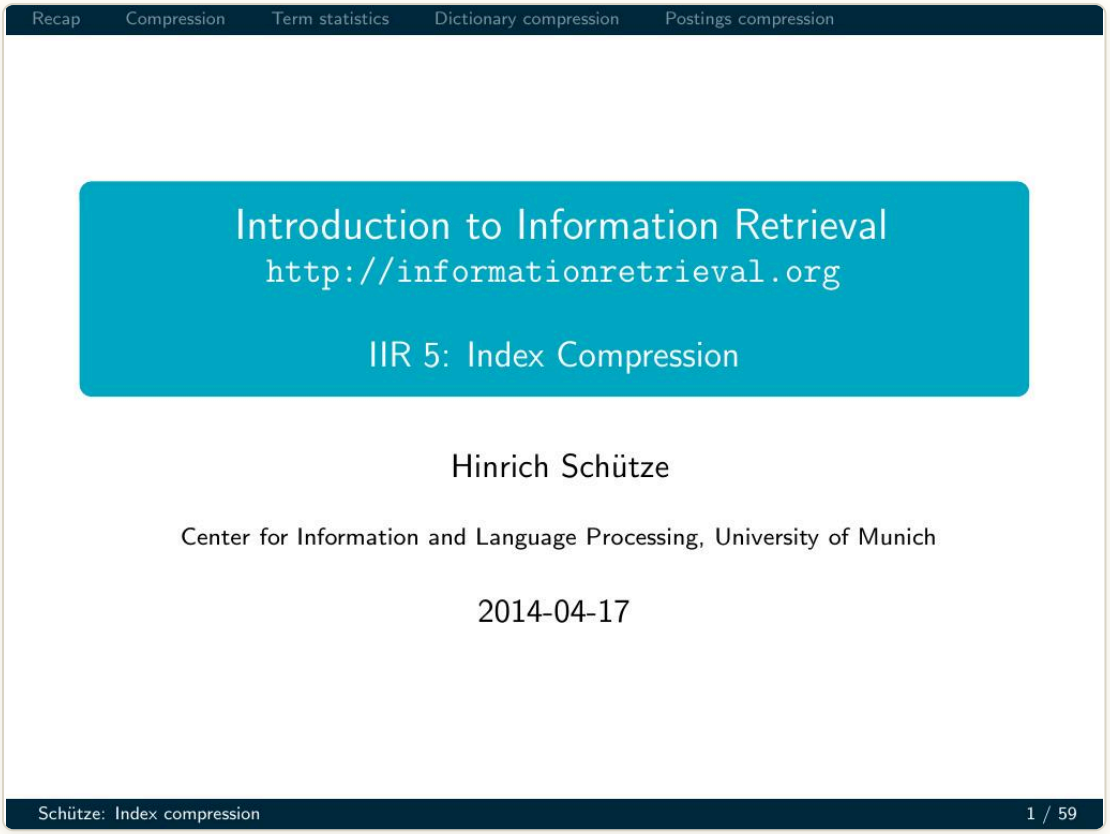
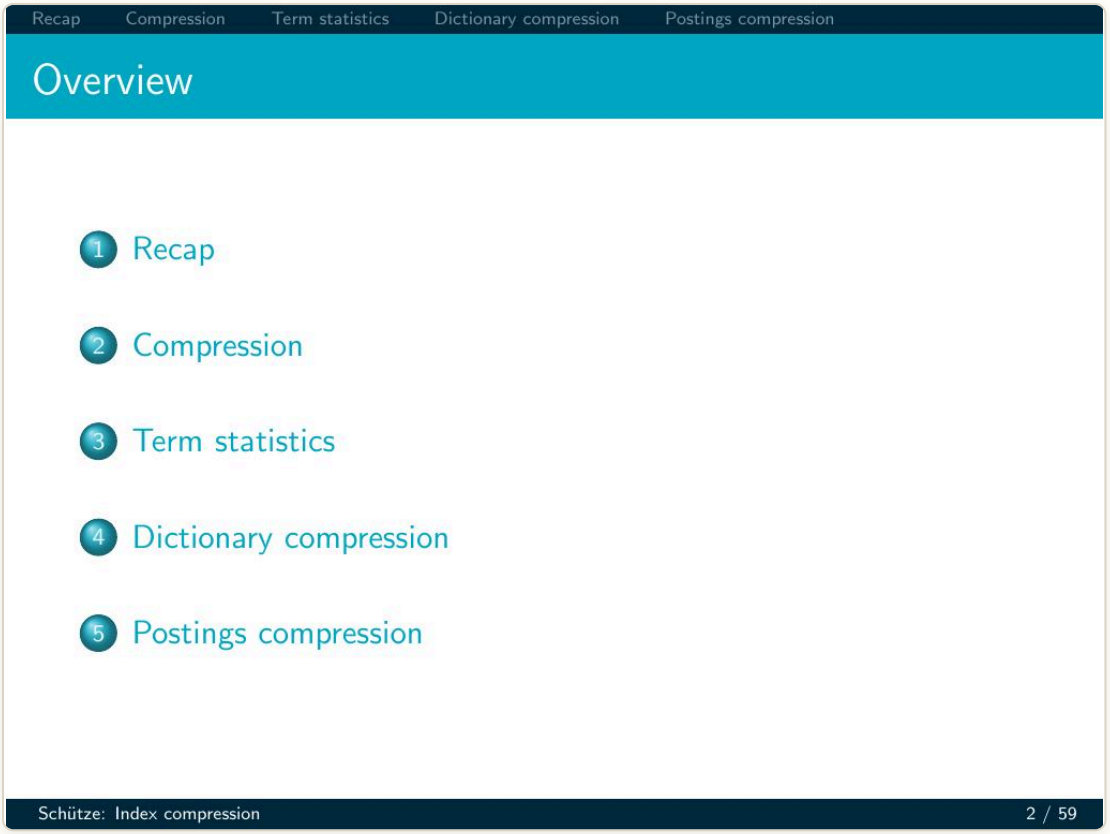


정보검색 입문 — IIR 5: 색인 압축

- Hinrich Schütze, 뮌헨대 (2014-04-17)

✍️ 메모

💡 이해 5강 핵심: 힙스 법칙(어휘 크기) · 지프 법칙(빈도 분포) → 사전 압축(문자열·블로킹·앞부분 코딩) → 포스팅 압축(간격 + VB 코드 / 감마 코드)

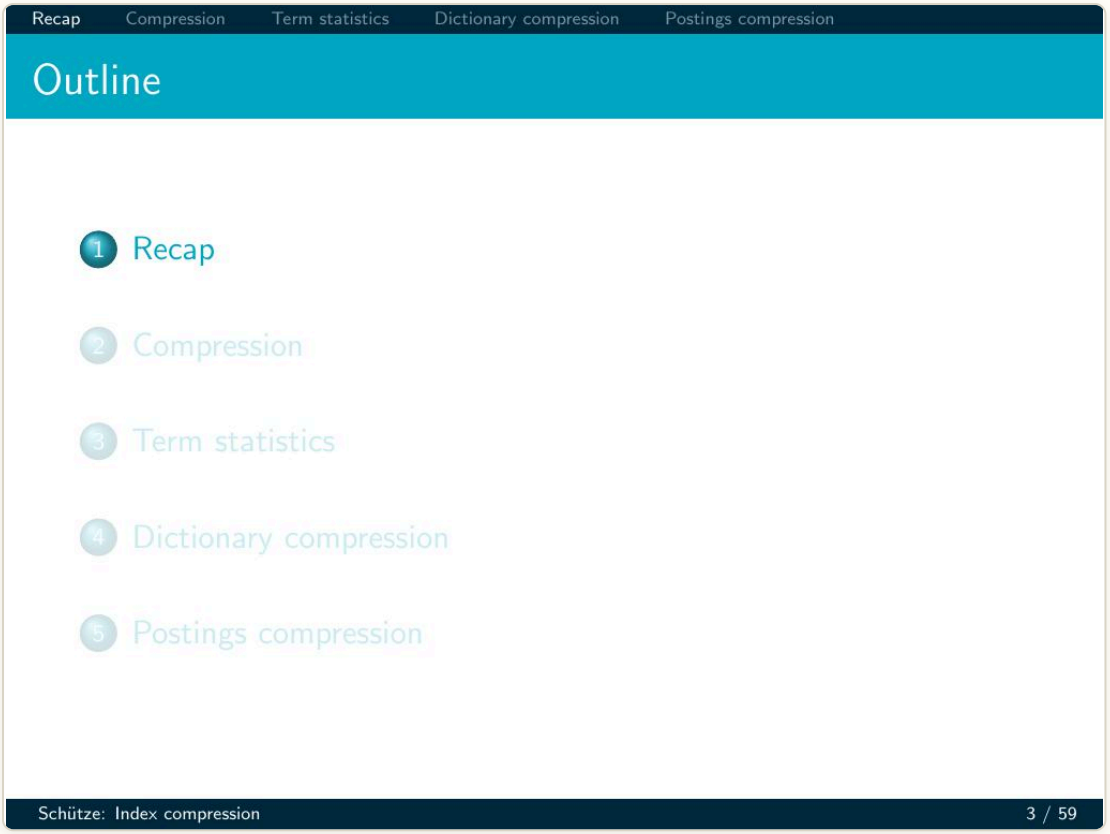


개요

- ① 복습 ② 압축 ③ 단어 통계 ④ 사전 압축 ⑤ 포스팅 압축

✍ 메모

— 넘어가도 되는 쪽

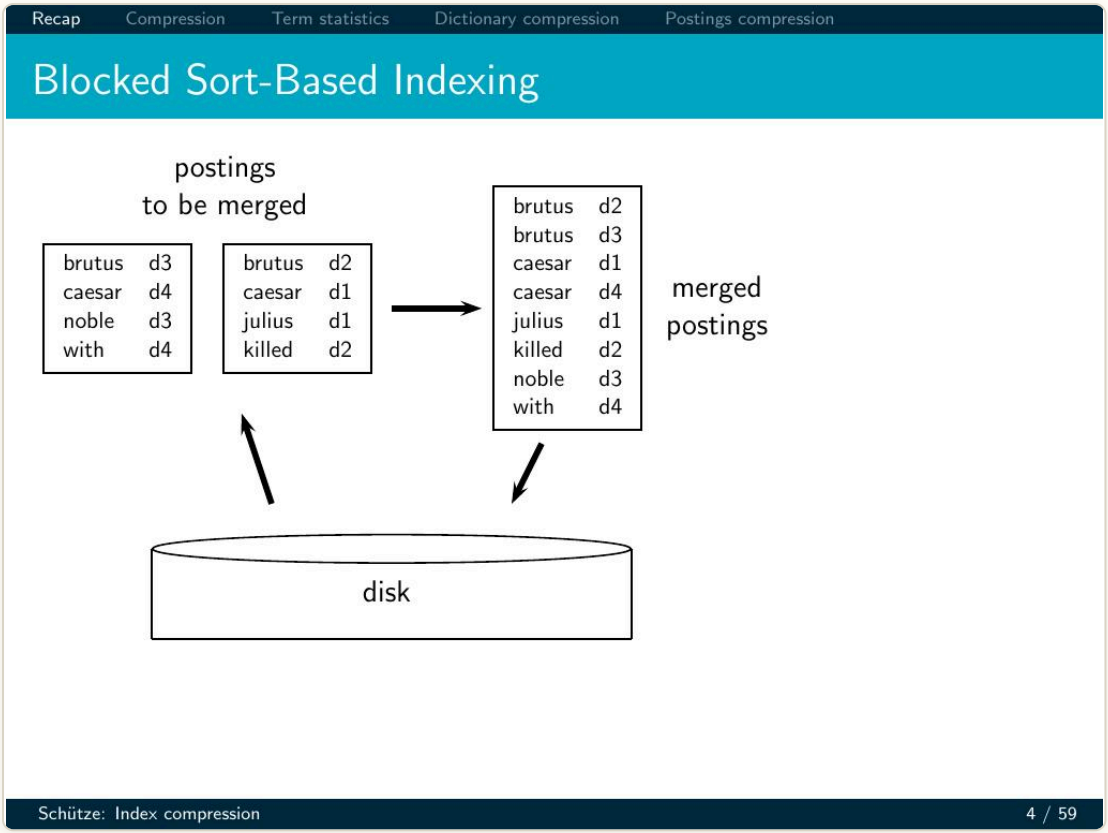


목차

- 복습 → 압축 → 단어 통계 → 사전 압축 → 포스팅 압축
- 지금: 복습

✍ 메모

— 넘어가도 되는 쪽

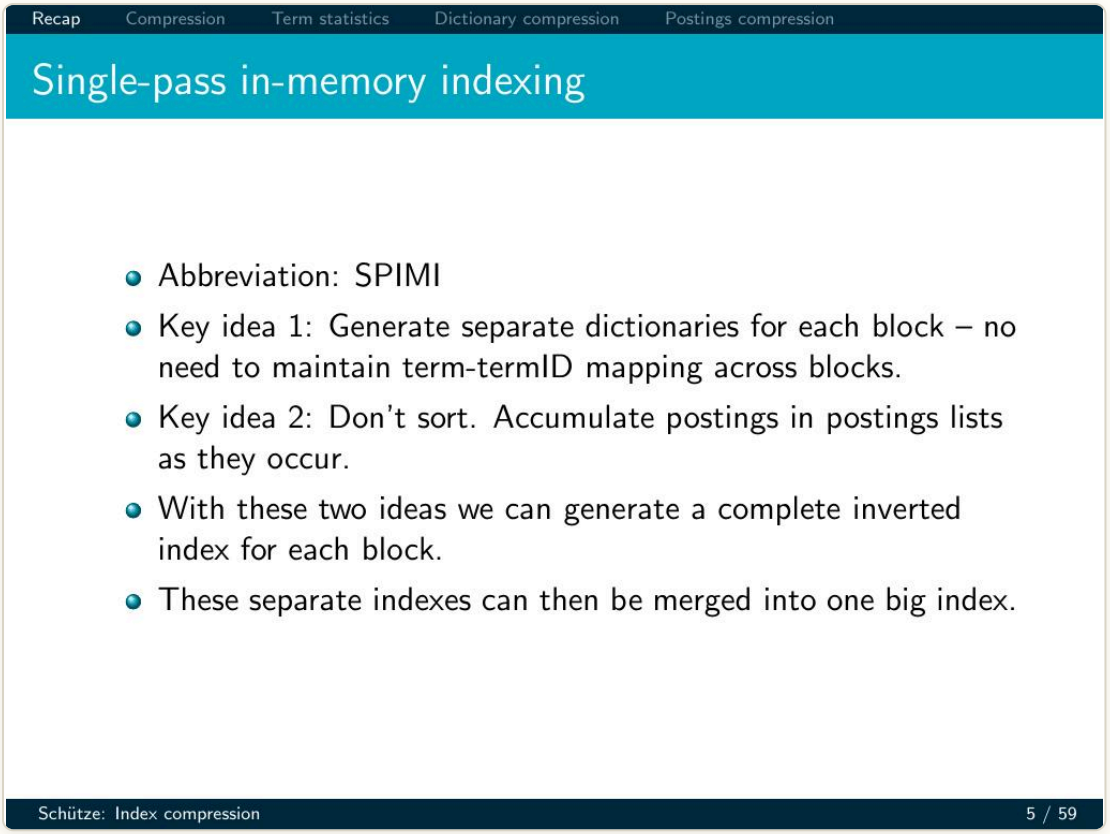


복습: 블록 정렬 기반 색인 (BSBI)

- 블록별로 정렬해 디스크에 쓰고, 블록들을 병합 (그림: Block 1 + Block 2 → 병합된 포스팅)

✍ 메모

— 넘어가도 되는 쪽

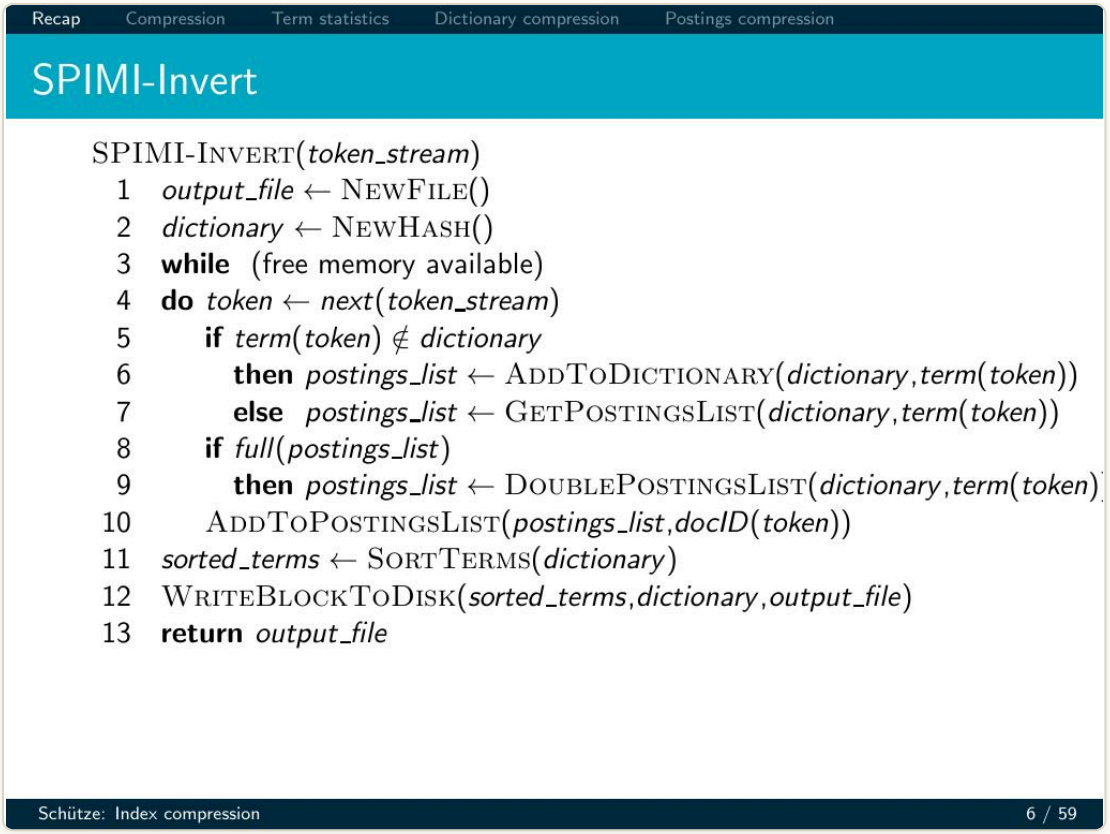


복습: 단일 패스 메모리 색인 (SPIMI)

- 핵심 1: 블록마다 별도 사전 — 블록 간 term-termID 대응 불필요
- 핵심 2: 정렬하지 않고 포스팅이 나오는 대로 쌓는다
- 블록마다 완전한 역색인 → 하나로 병합

✍ 메모

— 넘어가도 되는 쪽

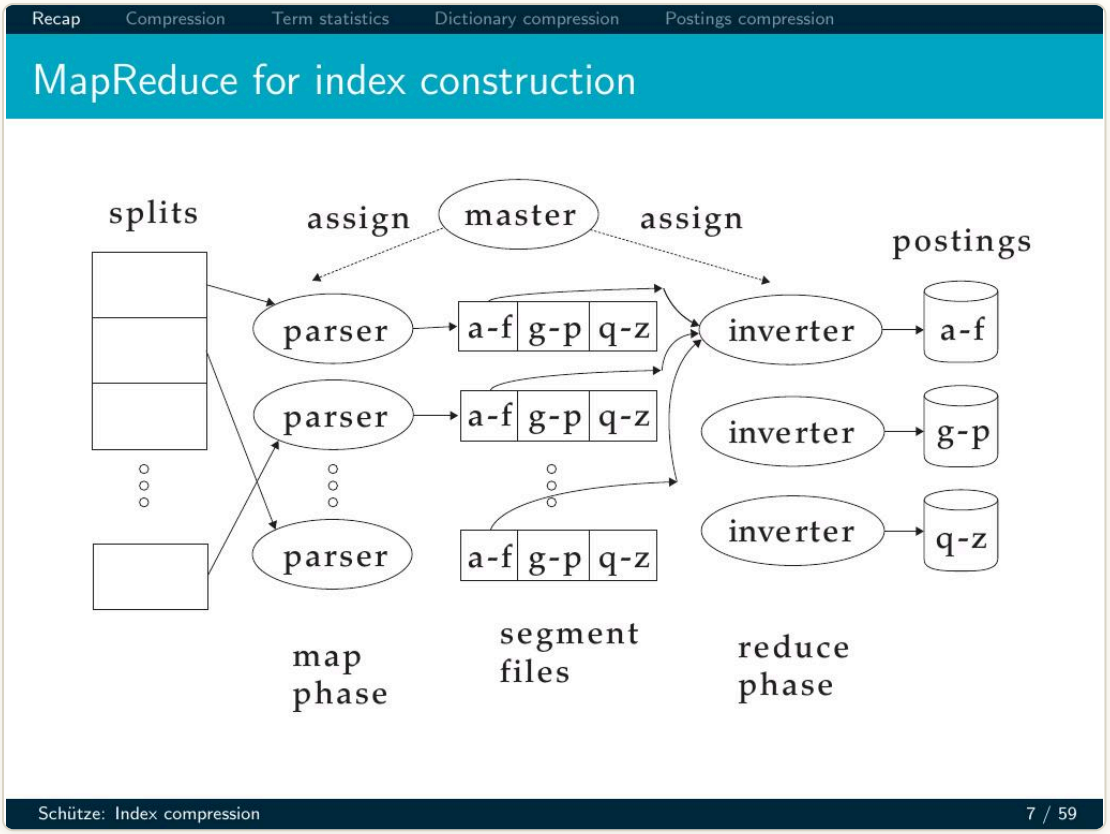


복습: SPIMI-INVERT

- 해시 사전에 토큰이 나오는 대로 포스팅 추가, 메모리가 차면 단어를 정렬해 블록을 디스크에 씀

✍ 메모

— 넘어가도 되는 쪽

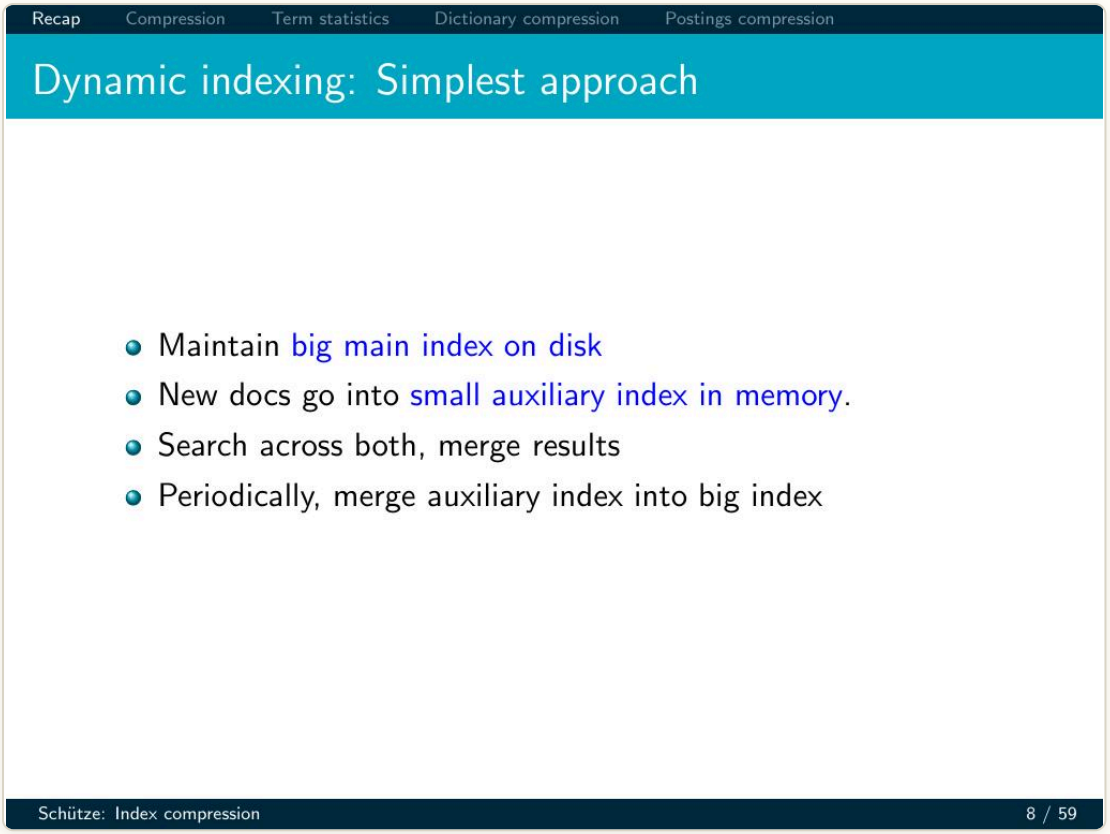


복습: 색인 구축용 MapReduce

- master → parser(map, a-f/g-p/q-z 세그먼트) → inverter(reduce, 포스팅)

✍ 메모

— 넘어가도 되는 쪽

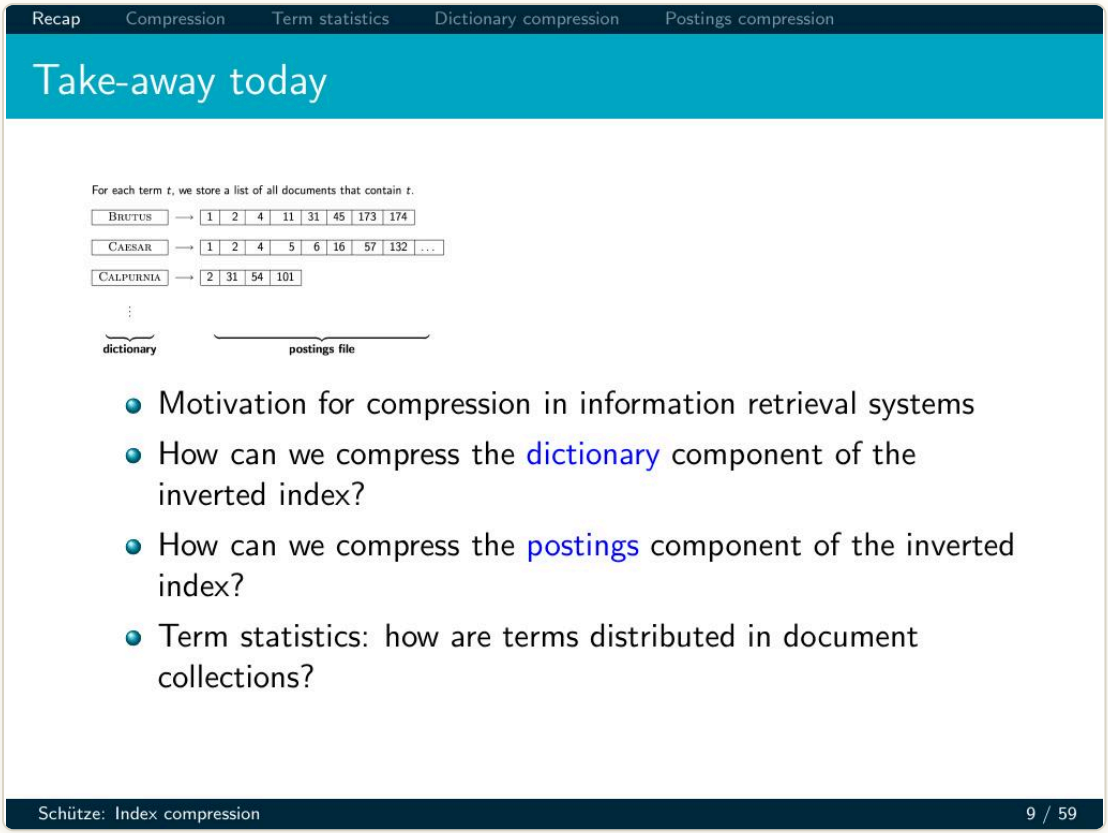


복습: 동적 색인 — 가장 단순한 방법

- 디스크에 큰 주 색인, 새 문서는 메모리의 작은 보조 색인
- 둘 다 검색해 결과 병합, 주기적으로 보조 색인을 주 색인에 병합

✍ 메모

— 넘어가도 되는 쪽

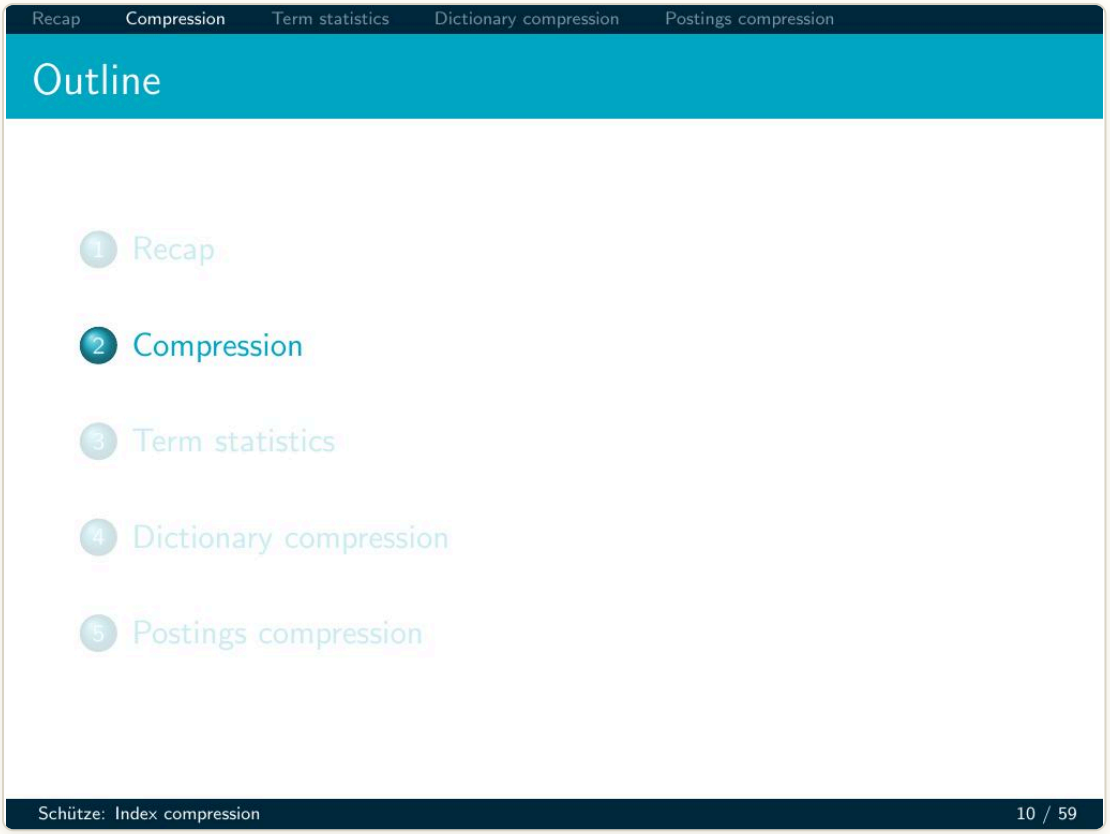


오늘 가져갈 것

- IR 시스템에서 **압축의 동기**
- 역색인의 **사전**을 어떻게 압축하나?
- 역색인의 **포스팅**을 어떻게 압축하나?
- **단어 통계**: 문서 컬렉션에서 단어는 어떻게 분포하나?

✍ 메모

— 넘어가도 되는 쪽

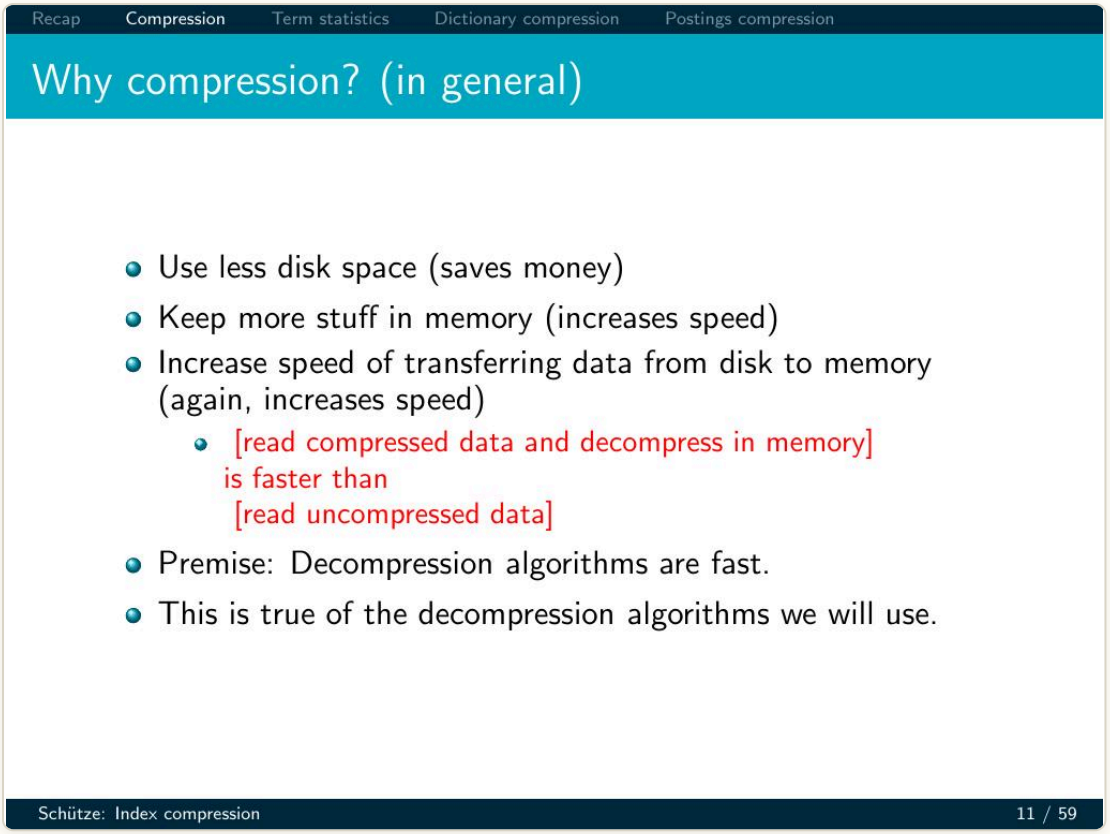


목차

- 복습 → 압축 → 단어 통계 → 사전 압축 → 포스팅 압축
- 지금: **압축**

✍️ 메모

— 넘어가도 되는 쪽

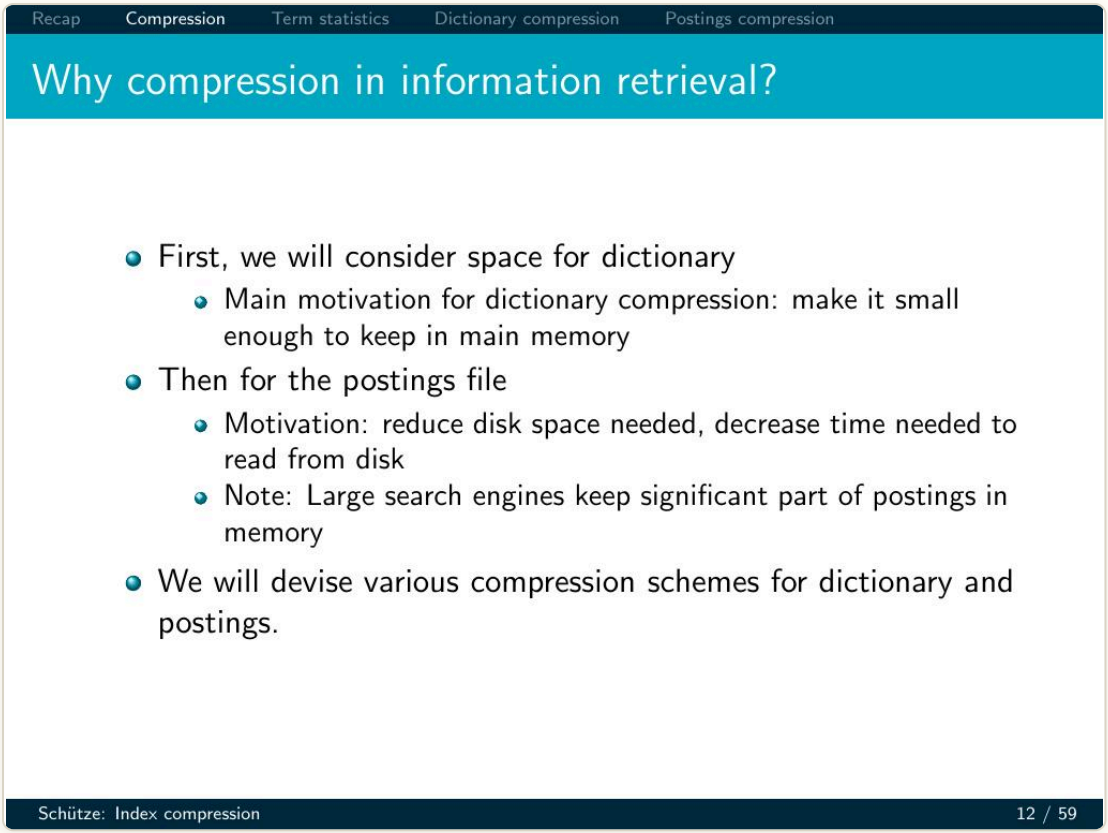


왜 압축하나? (일반적으로)

- **디스크 공간**을 덜 쓴다 (돈 절약)
- **메모리에 더 많이** 둔다 (속도↑)
- 디스크→메모리 **전송 속도** 증가 (속도↑)
  - [압축된 데이터를 읽고 메모리에서 해제] 가 [무압축 데이터 읽기] 보다 **빠르다**
- 전제: **해제 알고리즘이 빠르다** — 우리가 쓸 알고리즘은 그렇다

✍️ 메모

★ 시험 **압축 장점 3개: 디스크↓ · 메모리 캐시↑ · 전송 빨라짐**

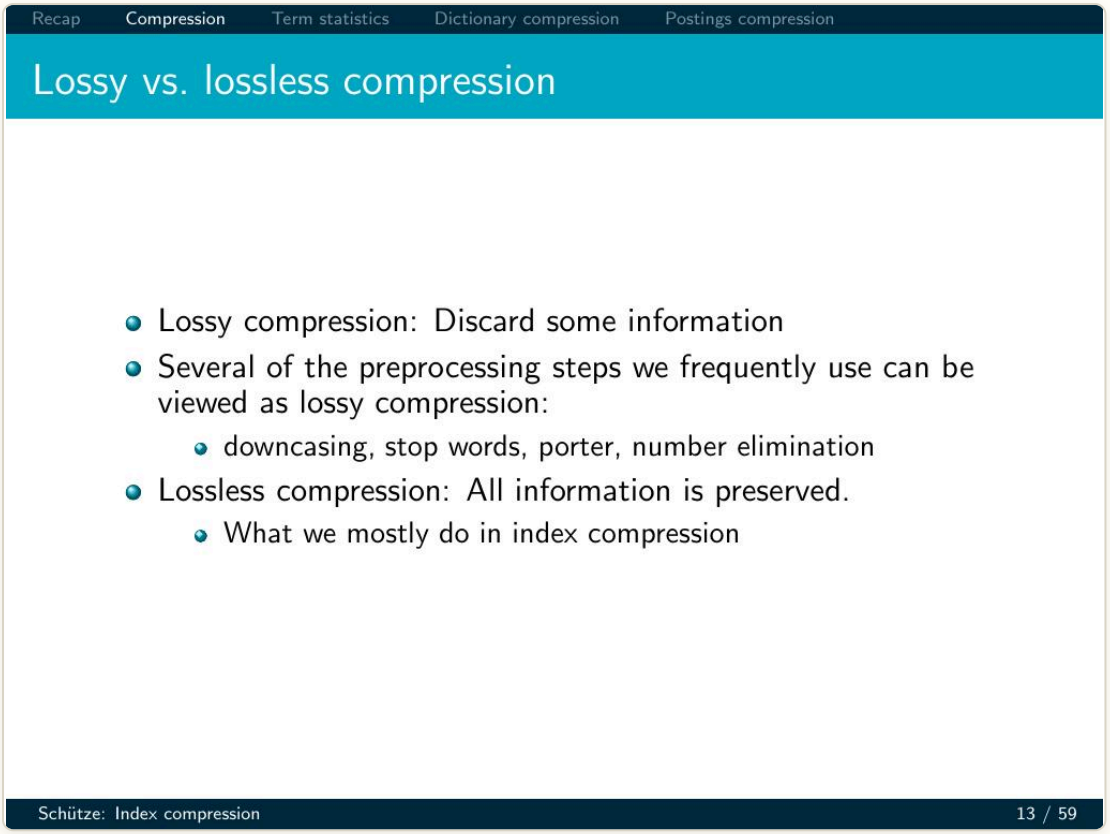


IR에서 왜 압축하나?

- 먼저 **사전**의 공간
  - 사전 압축의 주된 동기: **메인 메모리에 둘 만큼 작게**
- 그다음 **포스팅 파일**
  - 동기: 필요한 디스크 공간 줄이기, 디스크에서 읽는 시간 줄이기
- 참고: 대형 검색 엔진은 포스팅의 상당 부분을 **메모리**에 둔다
- 사전과 포스팅에 여러 압축 방식을 고안할 것

✍ 메모

★ 시험 사전 압축 = 메모리에 넣으려고 / 포스팅 압축 = 디스크·읽기 시간

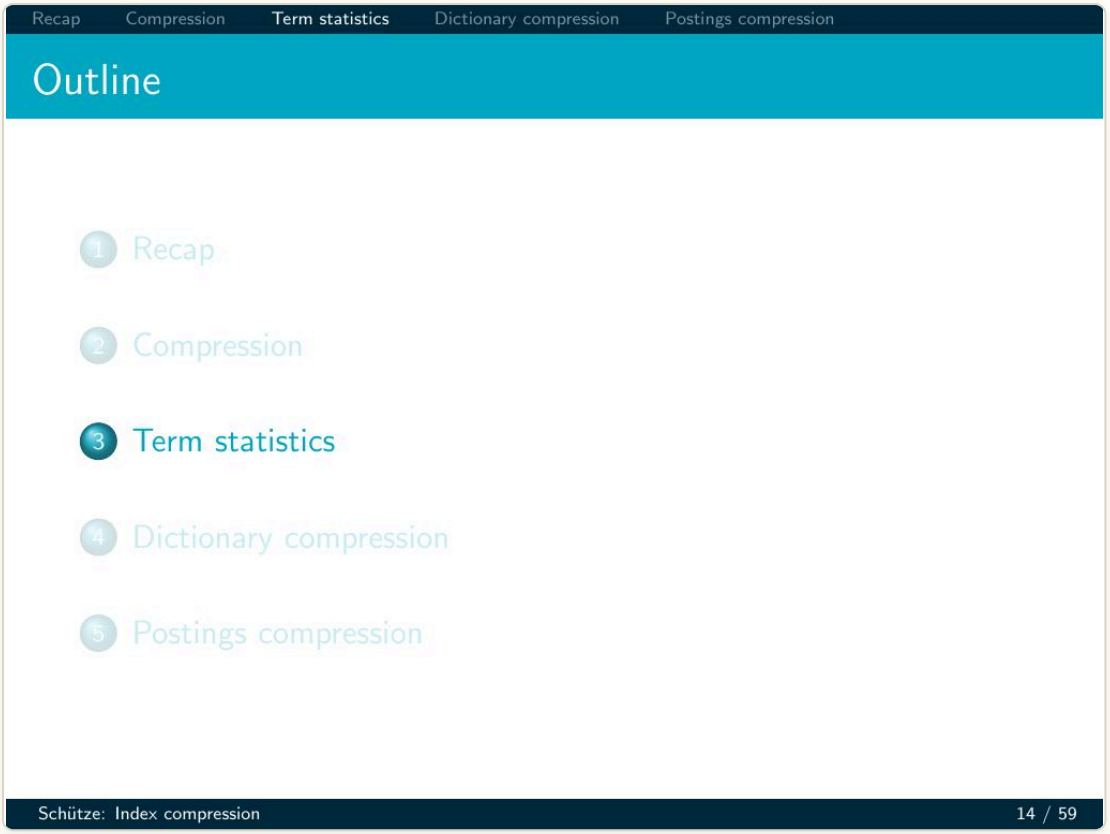


손실 vs 무손실 압축

- **손실(lossy) 압축**: 일부 정보를 버린다
  - 자주 쓰는 전처리 단계 여럿이 손실 압축으로 볼 수 있다:
  - 소문자화, 불용어, 포터 어간 추출, 숫자 제거
- **무손실(lossless) 압축**: 모든 정보를 보존
  - 색인 압축에서 주로 하는 것

✍️ 메모

★ 시험 전처리 = 손실 압축 / 색인 압축 = 무손실



목차

- 복습 → 압축 → 단어 통계 → 사전 압축 → 포스팅 압축
- 지금: **단어 통계**

✍ 메모

— 넘어가도 되는 쪽

Model collection: The Reuters collection

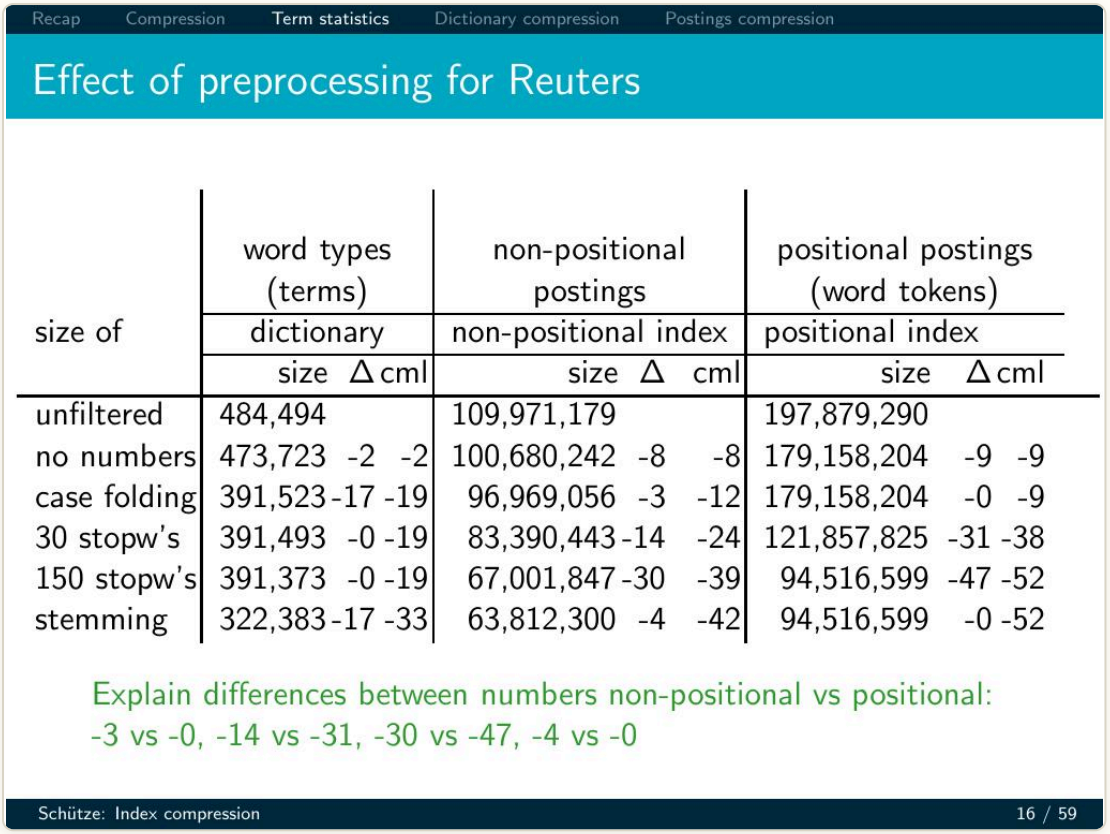
symbol	statistic	value
N	documents	800,000
L	avg. # word tokens per document	200
M	word types	400,000
	avg. # bytes per word token (incl. spaces/punct.)	6
	avg. # bytes per word token (without spaces/punct.)	4.5
	avg. # bytes per word type	7.5
T	non-positional postings	100,000,000

모델 컬렉션: 로이터

- N 문서 800,000 / L 문서당 토큰 200 / M 단어 유형 400,000
- 토큰당 바이트 6(공백 포함), 4.5(제외) / 유형당 바이트 7.5
- T 비위치 포스팅 100,000,000

✍ 메모

— 넘어가도 되는 쪽



로이터에서 전처리의 효과 (표)

- 단어 유형(사전) / 비위치 포스팅 / 위치 포스팅(토큰)의 크기와 감소율(Δ%, 누적 cml%)
  - 필터 없음: 484,494 / 109,971,179 / 197,879,290
  - 숫자 제거: 473,723 (−2, −2) / 100,680,242 (−8, −8) / 179,158,204 (−9, −9)
  - 대소문자 통일: 391,523 (−17, −19) / 96,969,056 (−3, −12) / 179,158,204 (−0, −9)
  - 불용어 30개: 391,493 (−0, −19) / 83,390,443 (−14, −24) / 121,857,825 (−31, −38)
  - 불용어 150개: 391,373 (−0, −19) / 67,001,847 (−30, −39) / 94,516,599 (−47, −52)
  - 어간 추출: 322,383 (−17, −33) / 63,812,300 (−4, −42) / 94,516,599 (−0, −52)
- 비위치 vs 위치 숫자 차이를 설명하라: −3 vs −0, −14 vs −31, −30 vs −47, −4 vs −0

✍ 메모

✏ 예·풀이 대소문자·어간: 같은 문서 안 포스팅만 합쳐짐(비위치↓), 토큰 수는 그대로(위치 −0)

✏ 예·풀이 불용어: 한 문서에 여러 번 나오는 흔한 단어라 토큰(위치)이 훨씬 많이 줄어든다

Recap Compression Term statistics Dictionary compression Postings compression

How big is the term vocabulary?

- That is, how many distinct words are there?
- Can we assume there is an upper bound?
- Not really: At least $70^{20} \approx 10^{37}$ different words of length 20.
- The vocabulary will keep growing with collection size.
- Heaps' law: $M = kT^b$
- M is the size of the vocabulary, T is the number of tokens in the collection.
- Typical values for the parameters k and b are: $30 \leq k \leq 100$ and $b \approx 0.5$.
- Heaps' law is linear in log-log space.
  - It is the simplest possible relationship between collection size and vocabulary size in log-log space.
  - Empirical law

Schütze: Index compression 17 / 59

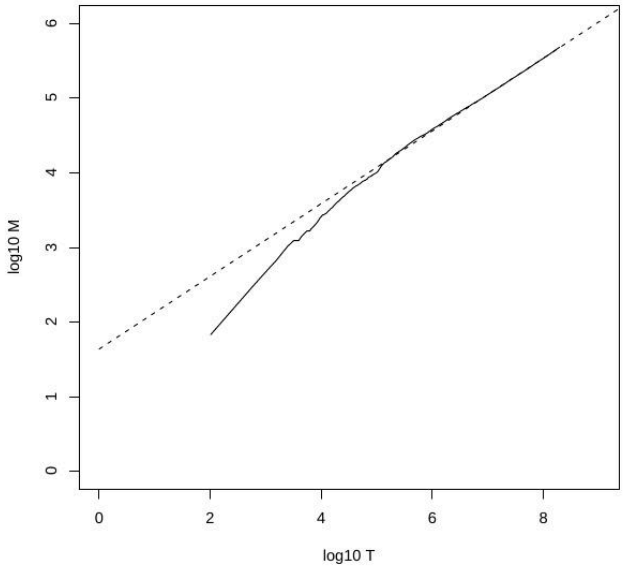
단어 어휘는 얼마나 클까?

- 서로 다른 단어가 몇 개나 될까? 상한이 있다고 볼 수 있나?
  - 아니다: 길이 20짜리 단어만 해도 최소 $70^{20} \approx 10^{37}$개
- 어휘는 컬렉션 크기와 함께 **계속 커진다**
- **힙스의 법칙 (Heaps' law): M = kT^b**
  - M = 어휘 크기, T = 컬렉션의 토큰 수
  - 보통 **30 ≤ k ≤ 100**, **b ≈ 0.5**
  - log-log 공간에서 **직선**
  - log-log 공간에서 컬렉션 크기와 어휘 크기의 가장 단순한 관계
  - **경험 법칙**

✍ 메모

★ 시험 Heaps: M = k·T^b (b ≈ 0.5 → 어휘는 토큰 수의 제곱근에 비례)

Heaps' law for Reuters



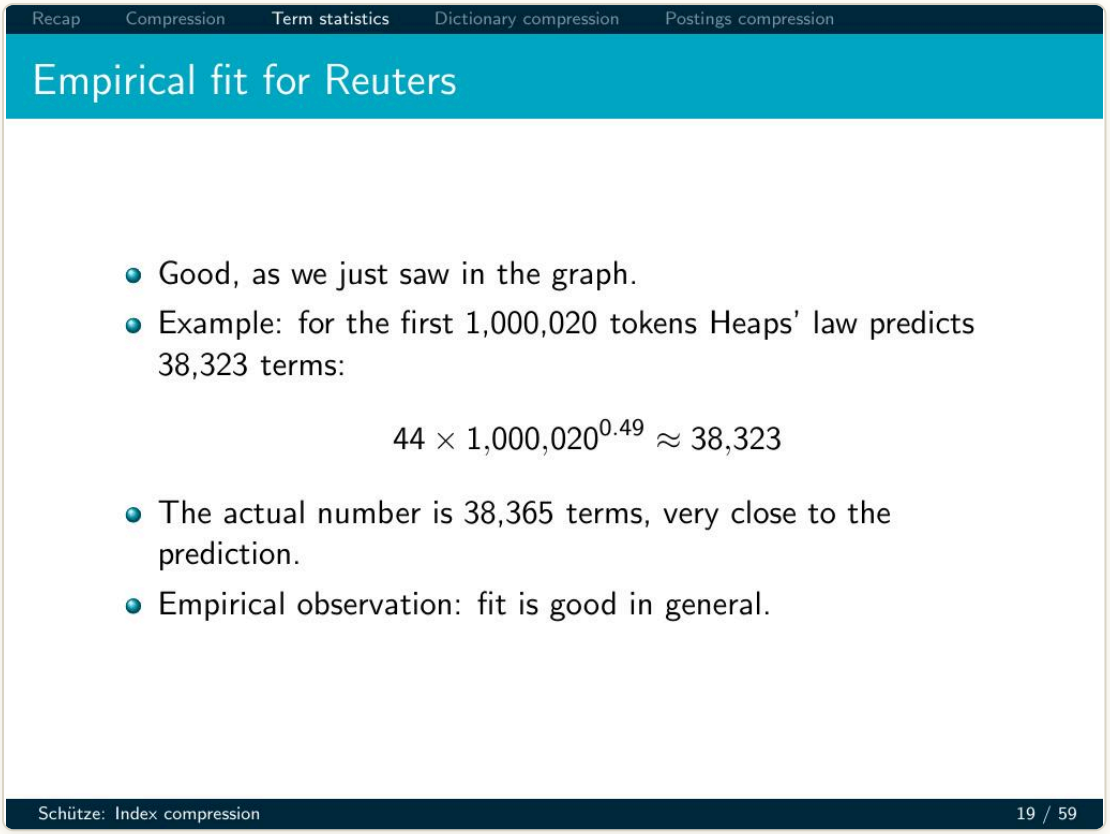
Vocabulary size M as a function of collection size T (number of tokens) for Reuters-RCV1. For these data, the dashed line $\log_{10} M = 0.49 * \log_{10} T + 1.64$ is the best least squares fit. Thus, $M = 10^{1.64} T^{0.49}$ and $k = 10^{1.64} \approx 44$ and $b = 0.49$.

로이터의 힙스 법칙 (그래프)

- 로이터 RCV1에서 컬렉션 크기 T(토큰 수)에 따른 어휘 크기 M
- 점선 $\log_{10}M = 0.49 \cdot \log_{10}T + 1.64$가 최소제곱 최적 직선
- 따라서 M = 10^1.64 · T^0.49 → **k = 10^1.64 ≈ 44**, **b = 0.49**

✍ 메모

— 넘어가도 되는 쪽

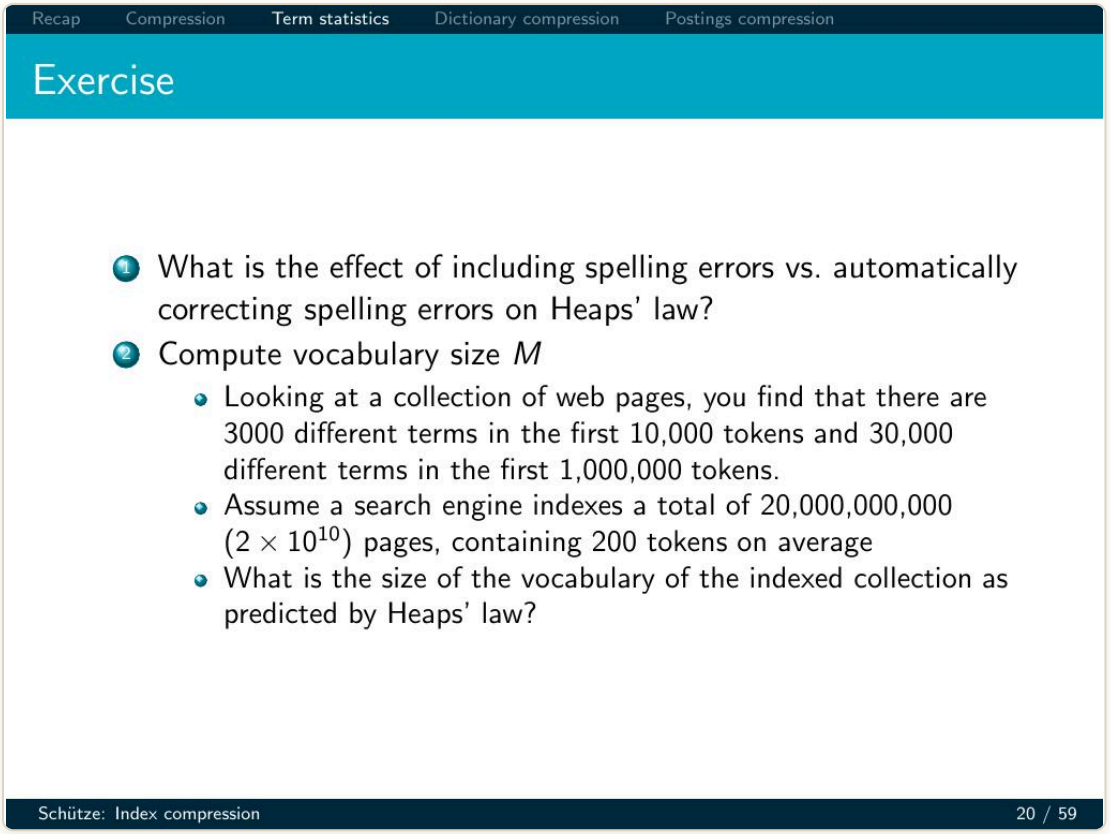


로이터에 대한 경험적 적합

- 그래프에서 봤듯 잘 맞는다
- 예: 처음 1,000,020 토큰에서 힙스 법칙 예측 **38,323**개 단어
  - 44 × 1,000,020^0.49 ≈ 38,323
- 실제는 **38,365**개 — 예측과 매우 가깝다
- 경험적으로 대체로 잘 맞는다

✍ 메모

— 넘어가도 되는 쪽



연습

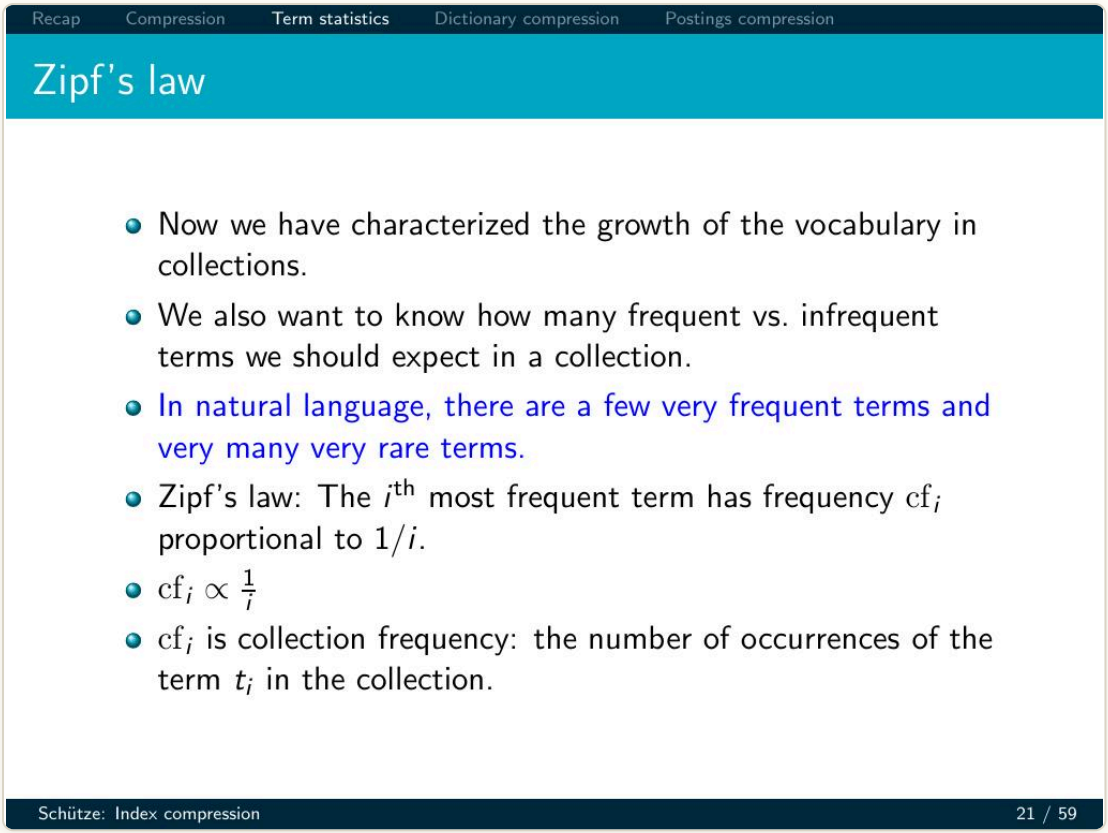
- ① 철자 오류를 교정하면 힙스 법칙에 어떤 영향이 있을까?
- ② 웹 페이지 컬렉션에서 처음 **10,000** 토큰에 서로 다른 단어 **3,000**개, 처음 **1,000,000** 토큰에 **30,000**개가 있다
  - 검색 엔진이 총 **2×10^{10}** 페이지(평균 **200** 토큰)를 색인한다면
  - 힙스 법칙이 예측하는 어휘 크기는?

✍ 메모

✏ 예·풀이 30,000/3,000 = $(10^6/10^4)$^b → 10 = 100^b → b = 0.5, k = 3000/√10000 = 30

✏ 예·풀이 T = $2\times10^{10} \times 200 = 4\times10^{12}$ → M = 30 × √(4×10^{12}) = 30 × 2×10^6 = **6천만(6×10^7)**

💡 이해 ①: 오타가 새 단어를 만들어 어휘를 키우므로, 교정하면 어휘 증가가 느려진다(k나 b 감소)

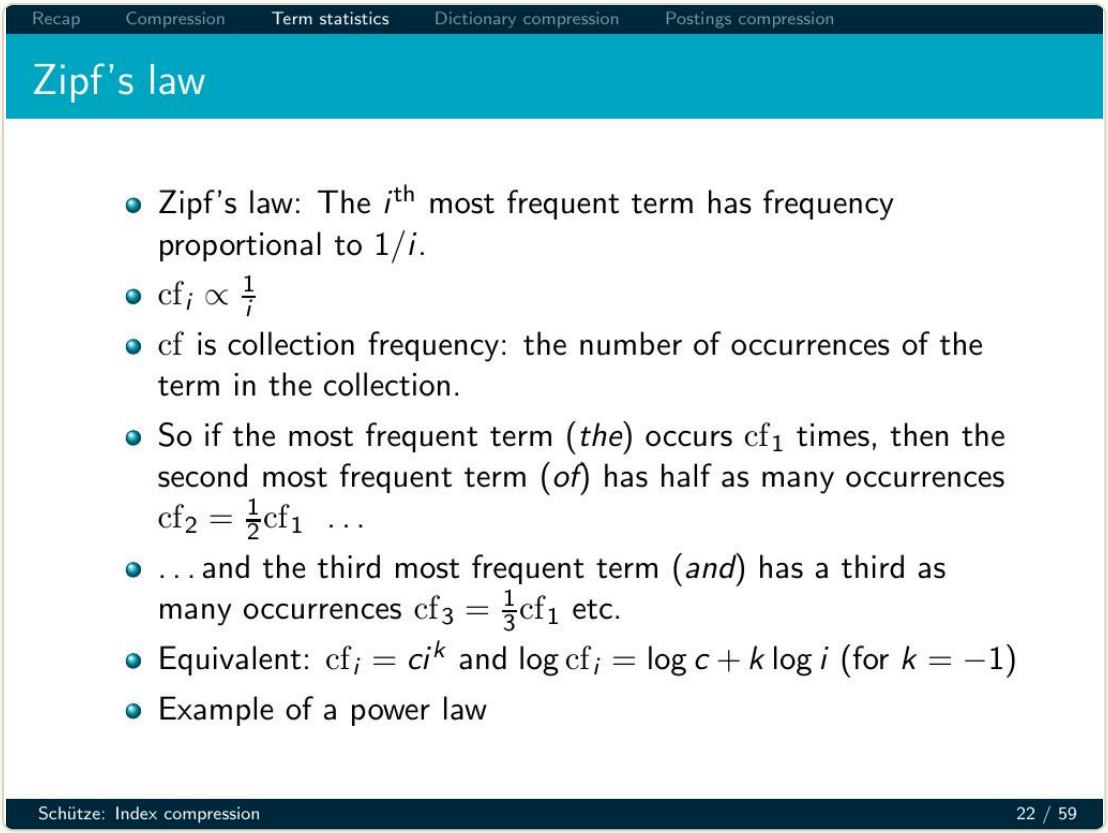


지프의 법칙 (Zipf's law)

- 컬렉션에서 어휘가 늘어나는 모양은 알았다
- 이제 **흔한 단어와 드문 단어**가 각각 얼마나 될지 알고 싶다
- 자연어엔 아주 흔한 단어 몇 개와 아주 드문 단어 **매우 많이**가 있다
- **지프의 법칙**: i번째로 흔한 단어의 빈도 **$cf_i \propto 1/i$**
- cf_i = **컬렉션 빈도**: 단어 t_i가 컬렉션에 나온 횟수

✍ 메모

★ 시험 **Zipf: cf_i ∝ 1/i (순위에 반비례)**



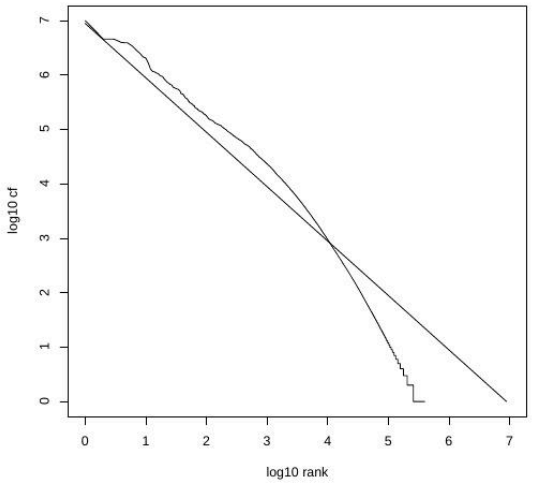
지프의 법칙 (계속)

- 가장 흔한 단어(the)가 cf_1번 나오면
  - 두 번째(of)는 그 **절반** $cf_2 = ½\ cf_1$ ...
  - 세 번째(and)는 **3분의 1** $cf_3 = ⅓\ cf_1$ 등
- 같은 말: $cf_i = c \cdot i^k$, $\log cf_i = \log c + k \log i$ ($k = -1$)
- **멱법칙(power law)**의 한 예

✍ 메모

⚠ 주의 Heaps = 어휘 크기 vs 토큰 수 / Zipf = 빈도 vs 순위 — 헷갈리지 말 것

Zipf's law for Reuters



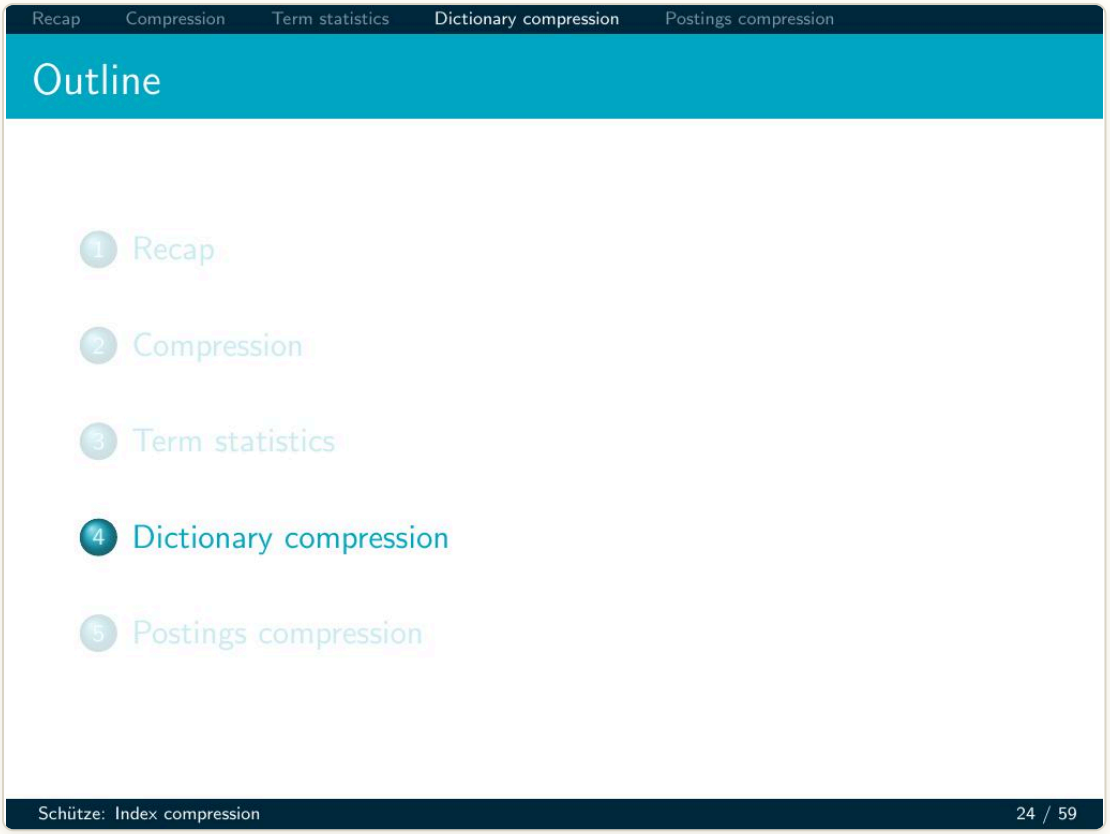
Fit is not great. What is important is the key insight: Few frequent terms, many rare terms.

로이터의 지프 법칙 (그래프)

- $\log_{10}$ 순위 vs $\log_{10}$ cf — 잘 맞진 않는다
- 중요한 건 핵심 통찰: **흔한 단어는 적고, 드문 단어는 많다**

✍ 메모

— 넘어가도 되는 쪽

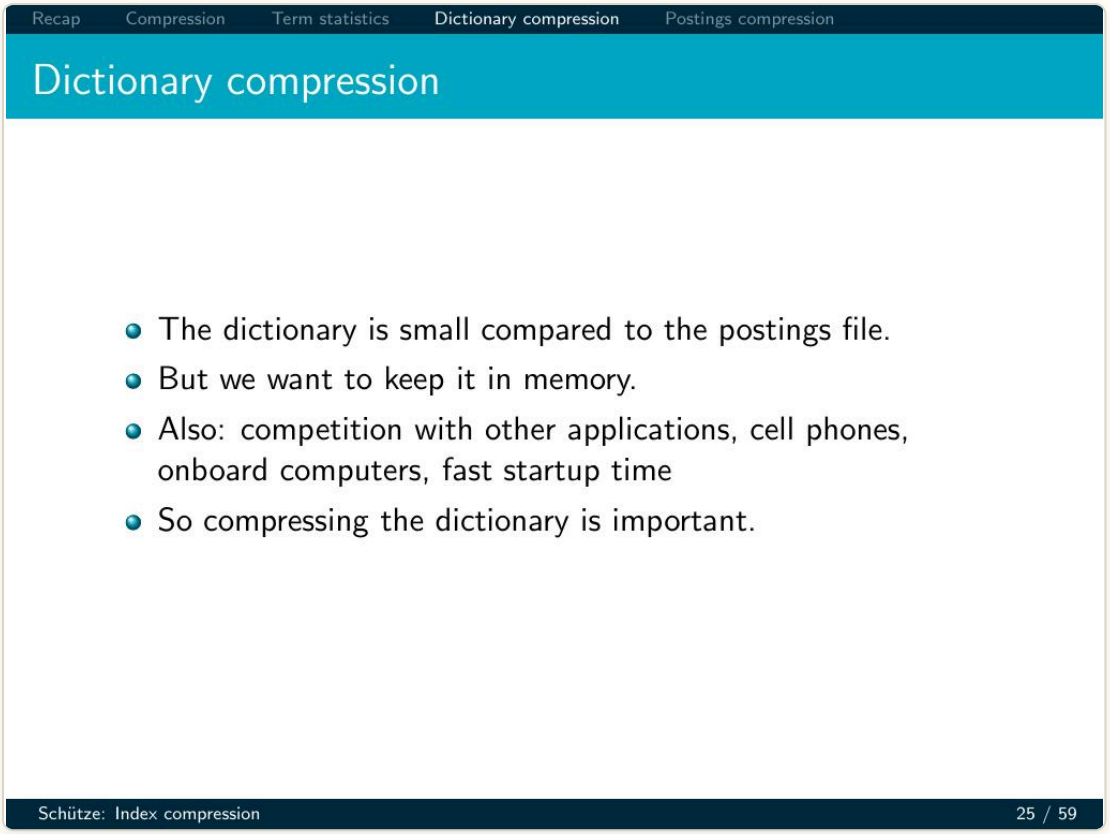


목차

- 복습 → 압축 → 단어 통계 → 사전 압축 → 포스팅 압축
- 지금: **사전 압축**

✍ 메모

— 넘어가도 되는 쪽



사전 압축

- 사전은 포스팅 파일에 비해 **작다**
- 하지만 **메모리에 두고** 싶다
- 또: 다른 응용과의 경쟁, 휴대폰, 탑재 컴퓨터, 빠른 시작 시간
- 그래서 사전 압축이 중요

✍ 메모

— 넘어가도 되는 쪽

Recall: Dictionary as array of fixed-width entries

	term	document frequency	pointer to postings list
	a	656,265	⟶
	aachen	65	⟶
	...	...	...
	zulu	221	⟶
space needed:	20 bytes	4 bytes	4 bytes

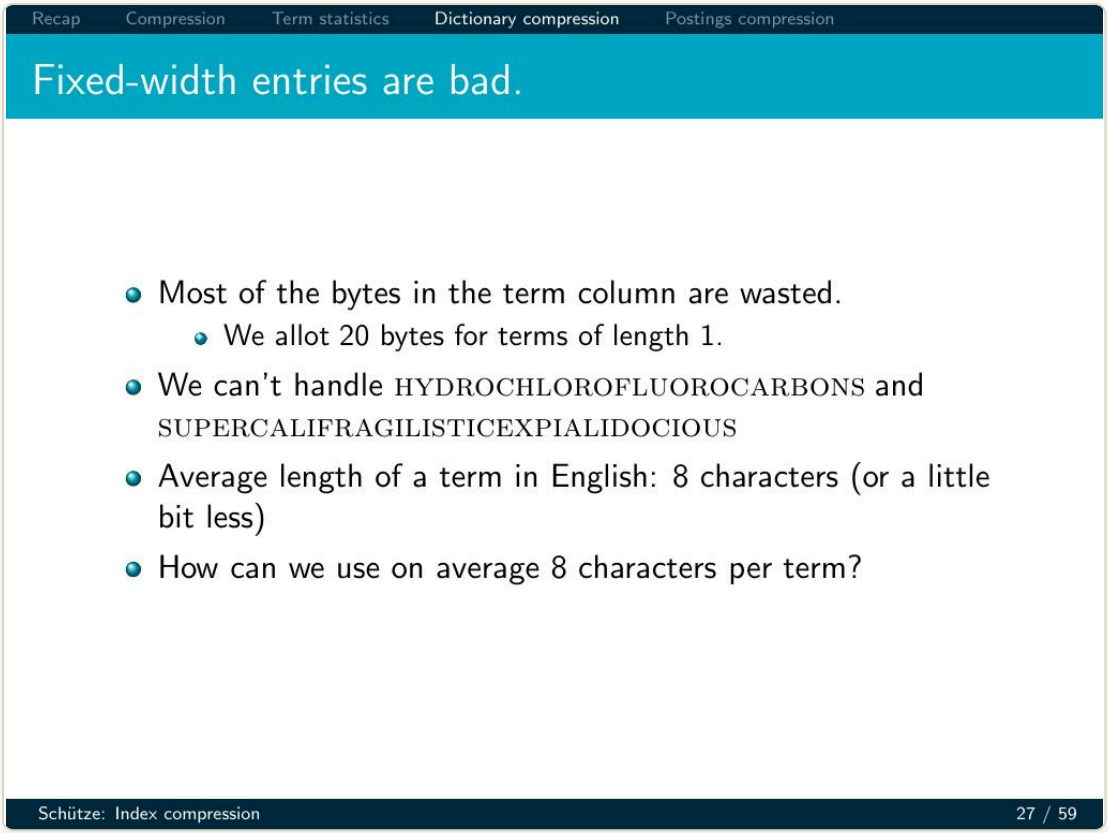
Space for Reuters: (20+4+4)*400,000 = 11.2 MB

복습: 고정 폭 항목 배열로서의 사전

- term 20바이트 · df 4바이트 · 포인터 4바이트
- 로이터 공간: (20 + 4 + 4) × 400,000 = **11.2MB**

✍ 메모

✏ 예·풀이 28바이트 × 40만 = 11.2MB

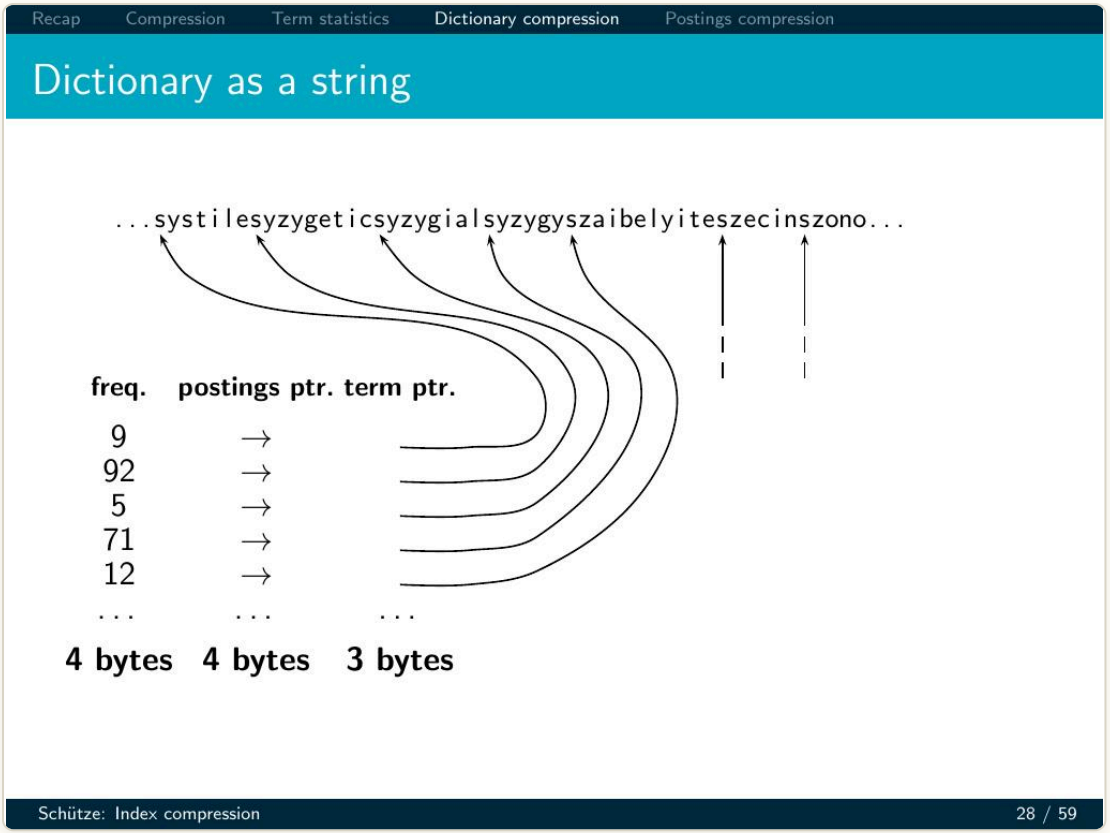


고정 폭 항목은 나쁘다

- 단어 열의 바이트 대부분이 **낭비**된다
  - 길이 1인 단어에도 20바이트를 준다
  - hydrochlorofluorocarbons나 supercalifragilisticexpialidocious는 **못 담는다**
- 영어 단어 평균 길이: **8글자** (또는 조금 적게)
- 평균 단어당 8글자만 쓰려면?

✍ 메모

— 넘어가도 되는 쪽

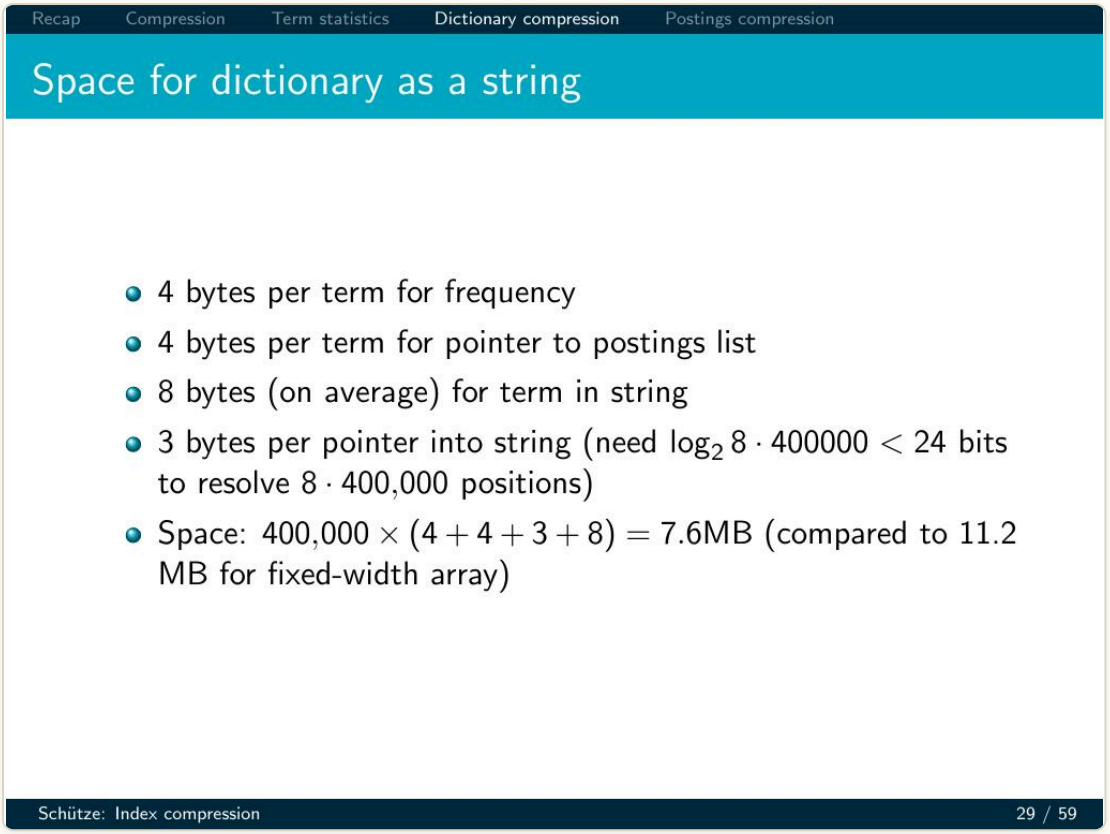


문자열로서의 사전

- 모든 단어를 **하나의 긴 문자열**로 이어 붙인다: ... systilesyzygeticsyzygialsyzygyszaibelyiteszecinszono...
- 각 항목은 df, 포스팅 포인터, 그리고 **문자열 안의 위치 포인터**를 가진다

✍ 메모

— 넘어가도 되는 쪽

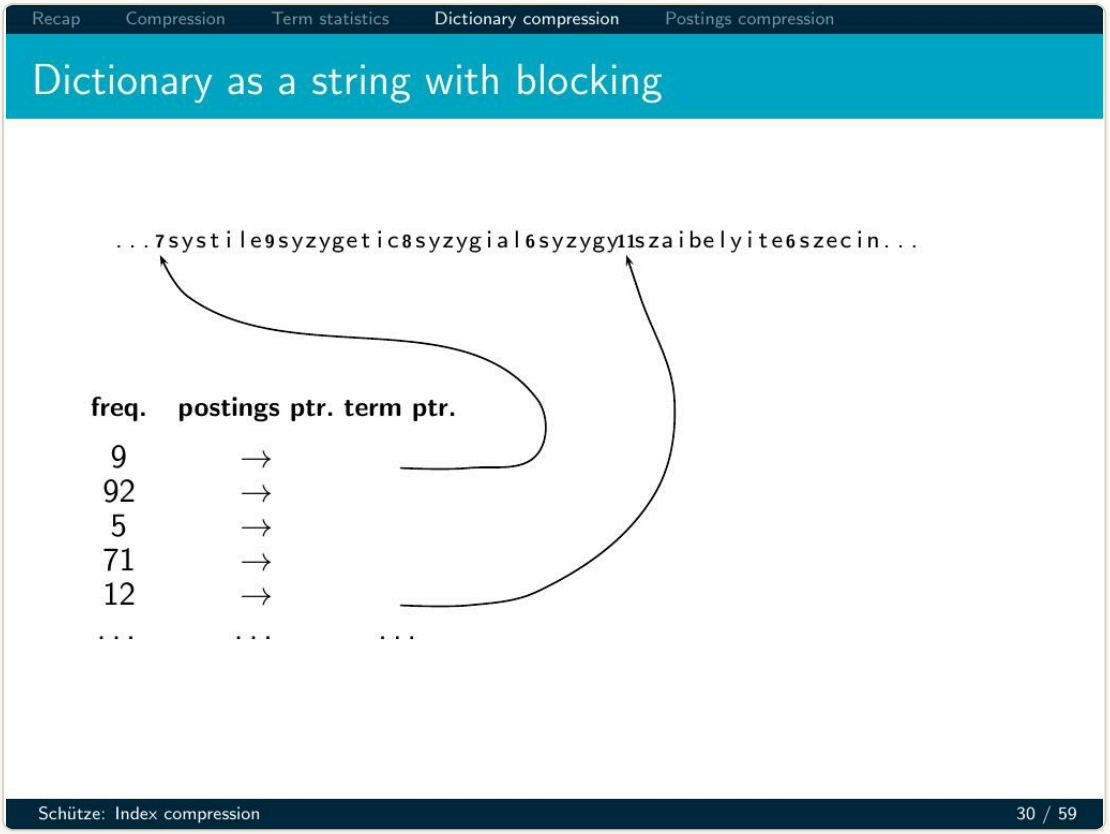


문자열 사전의 공간

- 단어당 빈도 **4바이트**
- 단어당 포스팅 포인터 **4바이트**
- 문자열 안 단어 **8바이트**(평균)
- 문자열 포인터 **3바이트** (8 × 400,000 위치를 구분하려면 $\log_2(8 \cdot 400000)$ < 24비트)
- 공간: 400,000 × (4 + 4 + 3 + 8) = **7.6MB** (고정 폭 배열은 11.2MB)

✍ 메모

★ 시험 계산식 그대로: 400,000 × 19 = 7.6MB

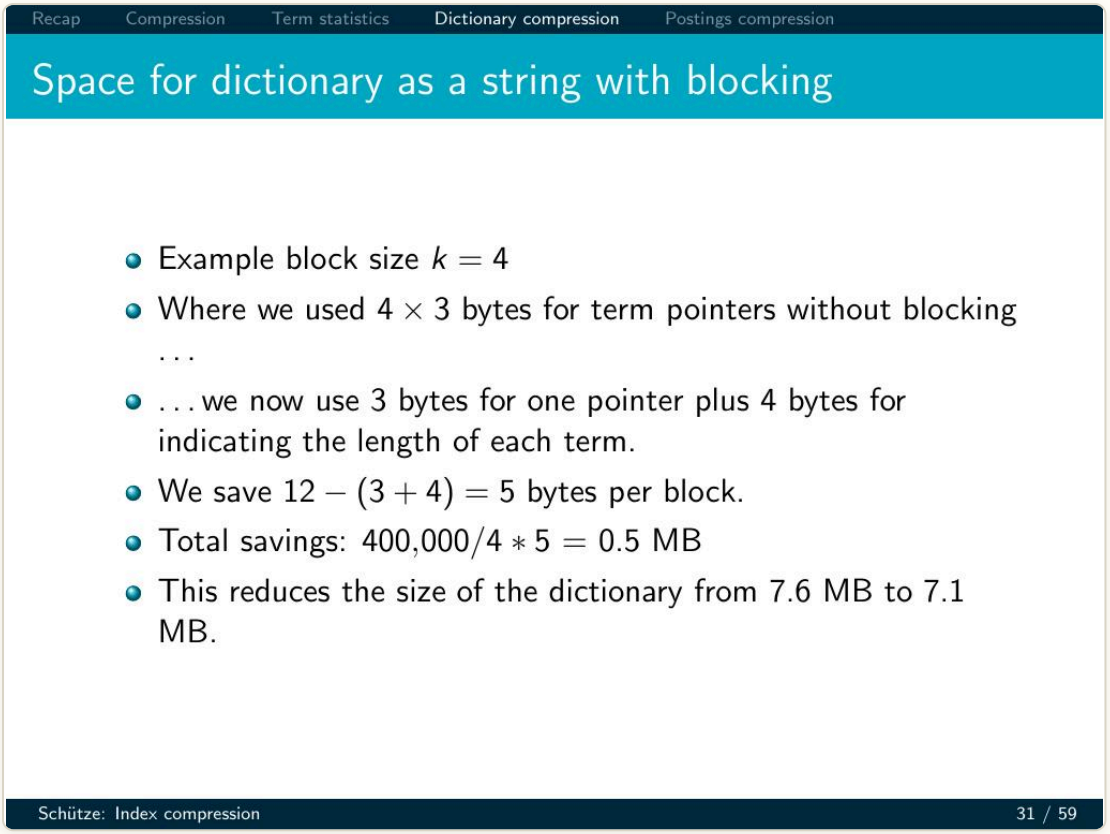


블로킹한 문자열 사전

- ...7systile9syzygetic8syzygial6syzygy11szaibelyite6szecin...
- 단어 앞에 **길이 1바이트**를 붙이고, **k개 단어마다** 문자열 포인터를 하나만 둔다

✍ 메모

— 넘어가도 되는 쪽

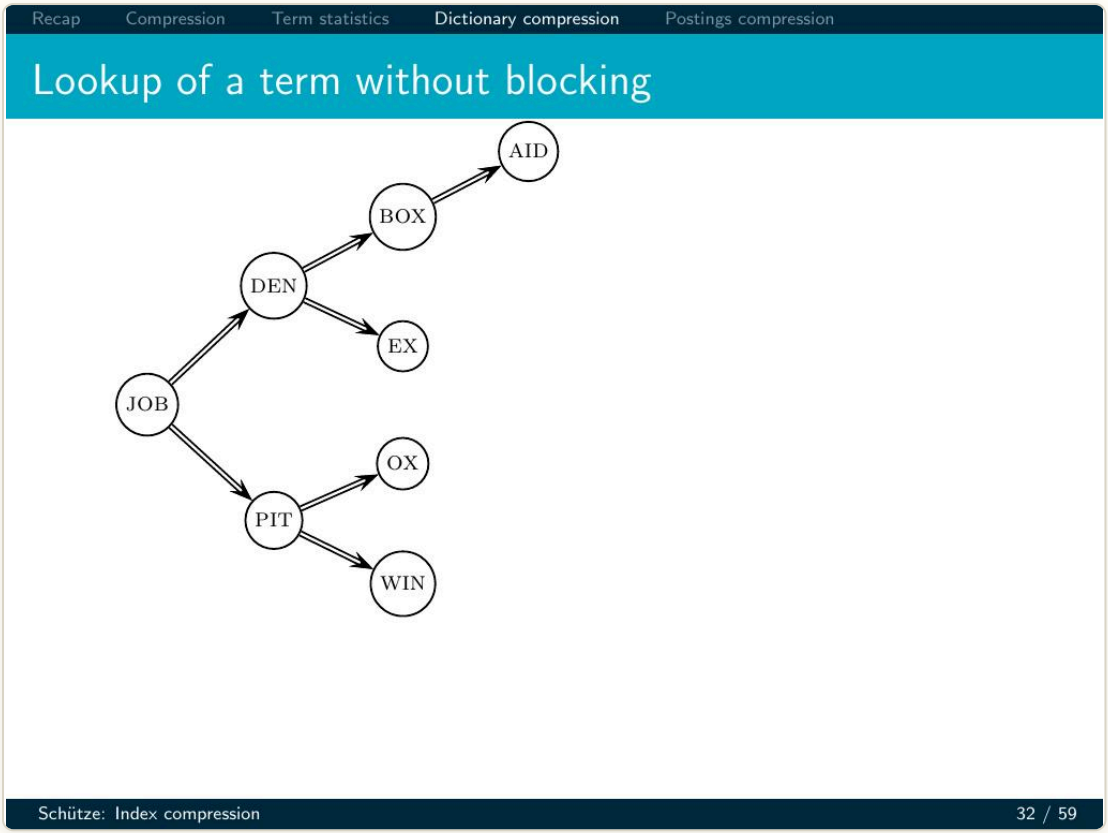


블로킹한 문자열 사전의 공간

- 예: 블록 크기 **k = 4**
- 블로킹 없이 단어 포인터에 4 × 3바이트를 썼다면 ...
- ... 이제 포인터 1개 **3바이트** + 각 단어 길이 표시 **4바이트**
- 블록당 12 − (3 + 4) = **5바이트** 절약
- 총 절약: 400,000 / 4 × 5 = **0.5MB**
- 사전 크기 7.6MB → **7.1MB**

✍️ 메모

★ 시험 블록당 5바이트 × 10만 블록 = 0.5MB

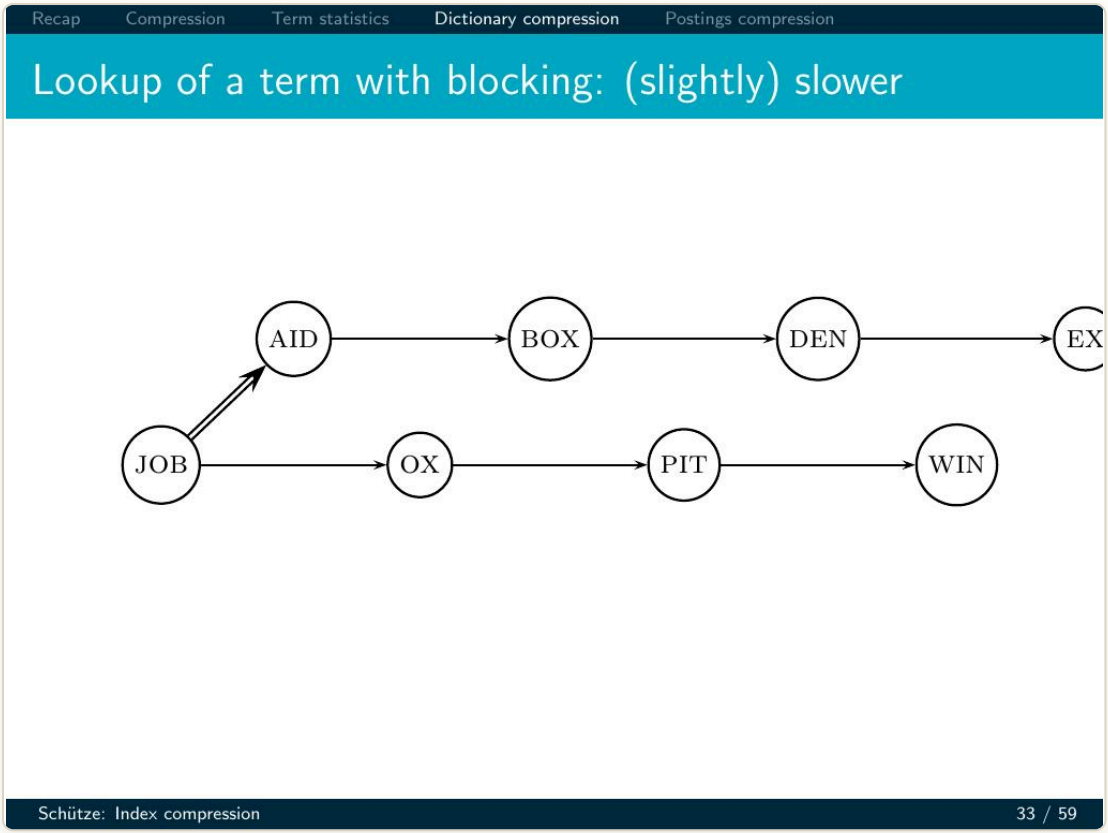


블로킹 없는 단어 조회 (그림)

- 이진 탐색 트리: JOB → DEN, PIT → BOX, EX, OX, WIN → AID ...
- 단어마다 포인터가 있어 바로 이진 탐색

✍️ 메모

— 넘어가도 되는 쪽

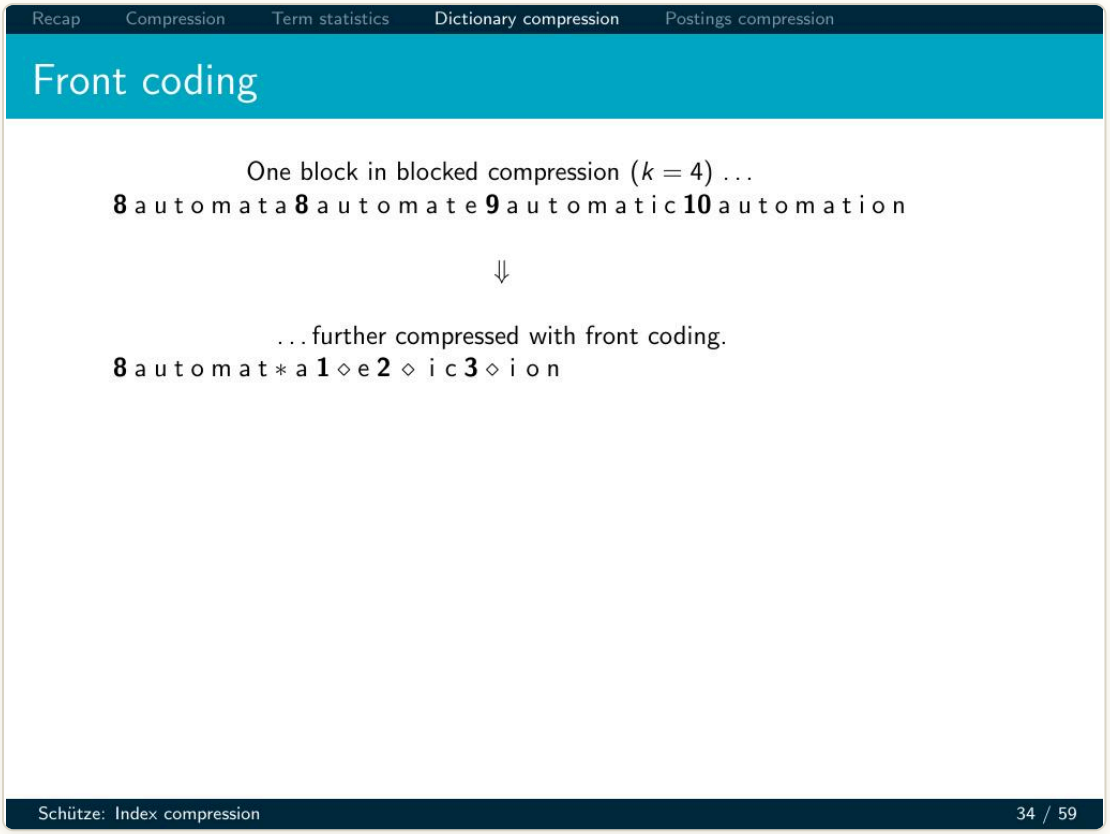


블로킹한 단어 조회: (약간) 느리다

- 블록(4개 단어) 단위로 이진 탐색한 뒤, 블록 안에서는 **선형 탐색**
- 그림: JOB → AID → BOX → DEN → EX / JOB → OX → PIT → WIN

✍ 메모

★ 시험 블로킹 = 공간↓, 조회 약간 느려짐 (블록 안 선형 탐색)

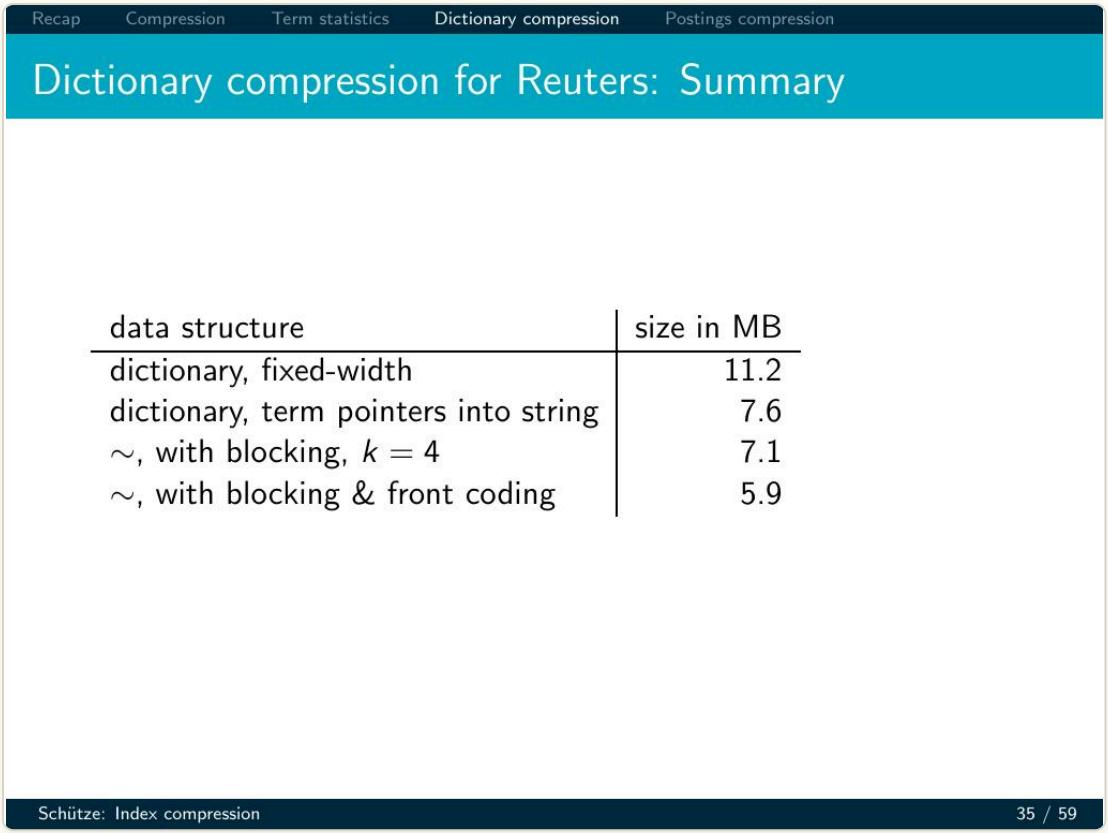


앞부분 코딩 (Front coding)

- 블로킹 압축의 블록 하나 (k = 4)
  - 8automata 8automate 9automatic 10automation
  - ⇓ 앞부분 코딩으로 더 압축
  - **8automat⋆a1⋄e2⋄ic3⋄ion**
- 공통 접두어 automat을 한 번만 쓰고(* 표시), 나머지는 남는 길이와 접미어만 (⋄ = 접두어 자리)

✍ 메모

★ 시험 정렬된 단어는 앞부분이 겹친다 → 공통 접두어를 한 번만

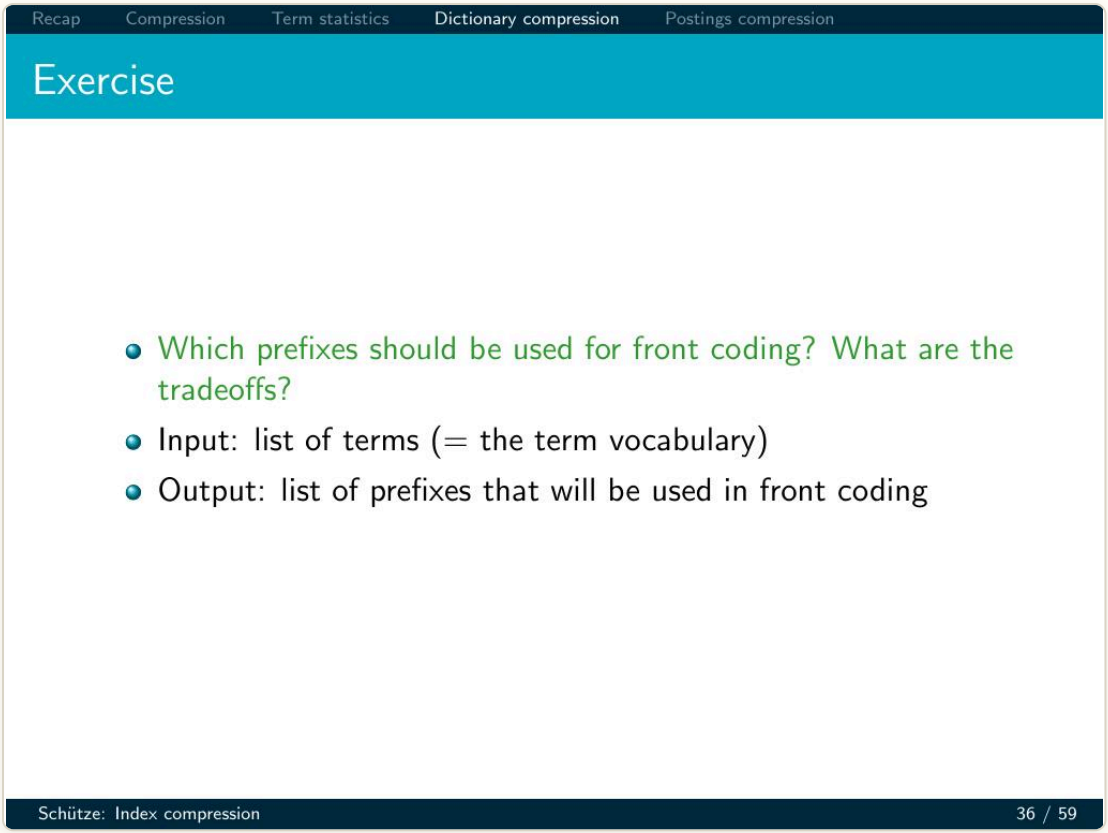


로이터 사전 압축 요약

- 사전, 고정 폭: **11.2MB**
- 사전, 문자열 안 단어 포인터: **7.6MB**
  - + 블로킹 k = 4: **7.1MB**
  - + 블로킹 & 앞부분 코딩: **5.9MB**

✍️ 메모

★ 시험 11.2 → 7.6 → 7.1 → 5.9 순서 외우기

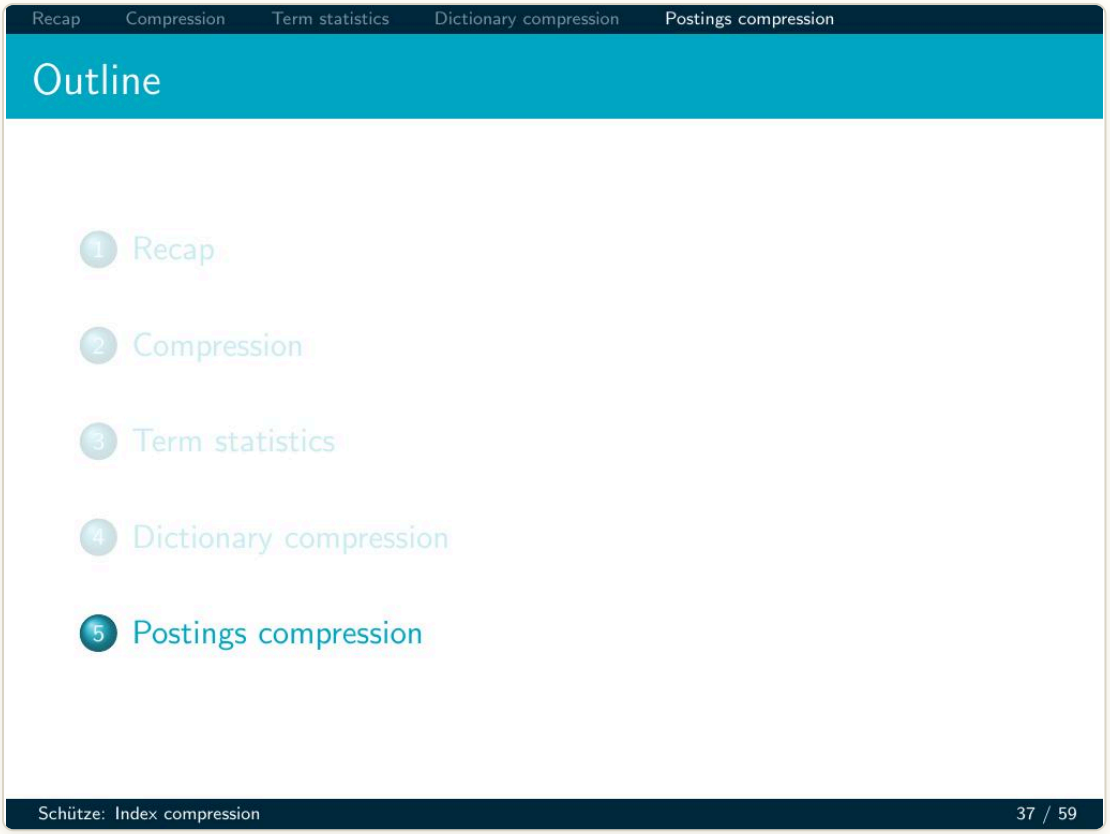


연습

- 앞부분 코딩에 **어떤 접두어**를 쓸까? 절충점은?
- 입력: 단어 목록 (= 단어 어휘)
- 출력: 앞부분 코딩에 쓸 접두어 목록

✍️ 메모

💡 이해 긴 공통 접두어 = 압축↑ but 블록 경계·조회 복잡도와 절충

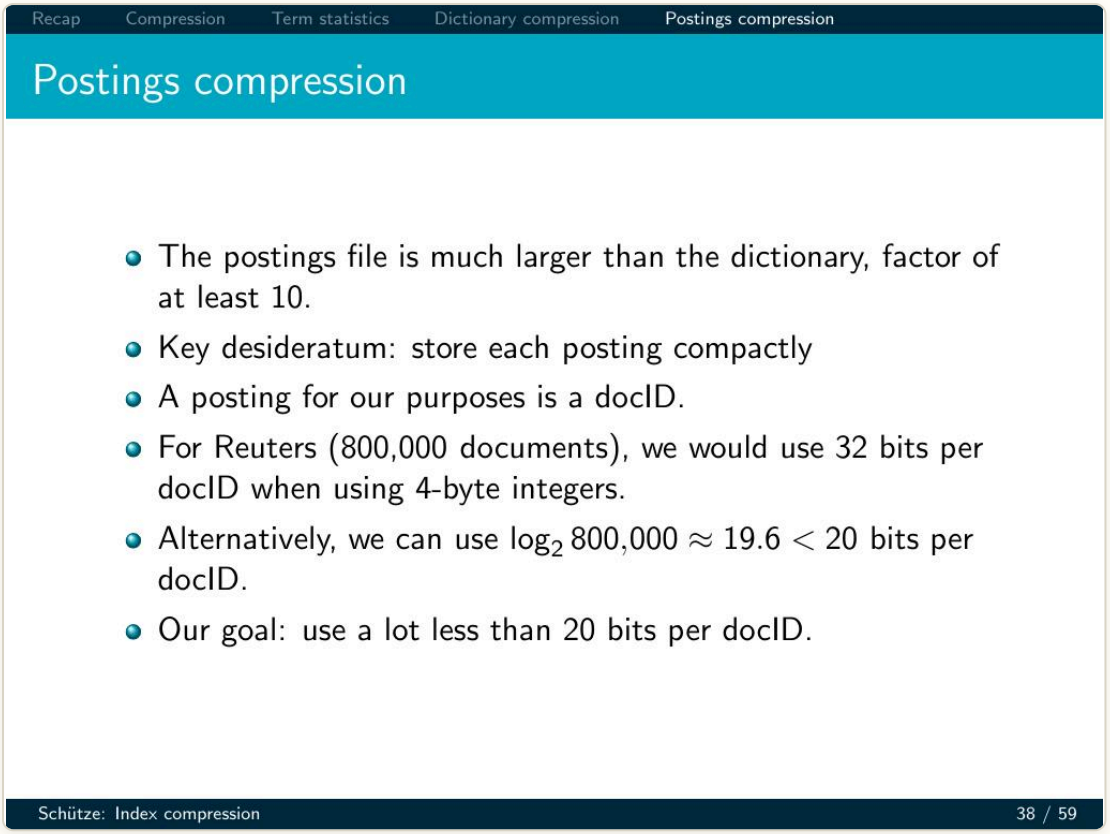


목차

- 복습 → 압축 → 단어 통계 → 사전 압축 → 포스팅 압축
- 지금: **포스팅 압축**

✍ 메모

— 넘어가도 되는 쪽

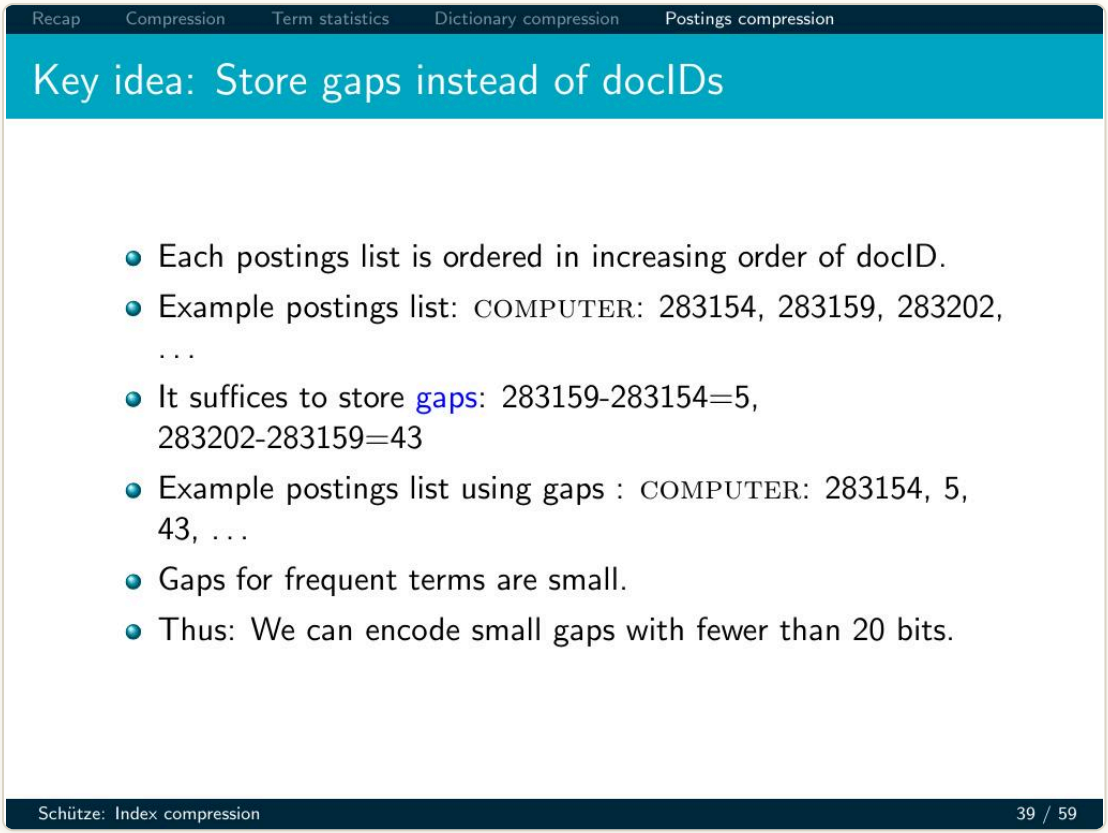


포스팅 압축

- 포스팅 파일은 사전보다 훨씬 크다 — **최소 10배**
- 핵심 요구: 각 포스팅을 **촘촘하게** 저장
- 여기서 포스팅 = **docID**
- 로이터(문서 800,000개)에서 4바이트 정수를 쓰면 docID당 **32비트**
- 대신 $\log_2 800{,}000 \approx 19.6 <$ **20비트**를 쓸 수도
- 목표: docID당 20비트보다 **훨씬 적게**

✍ 메모

— 넘어가도 되는 쪽

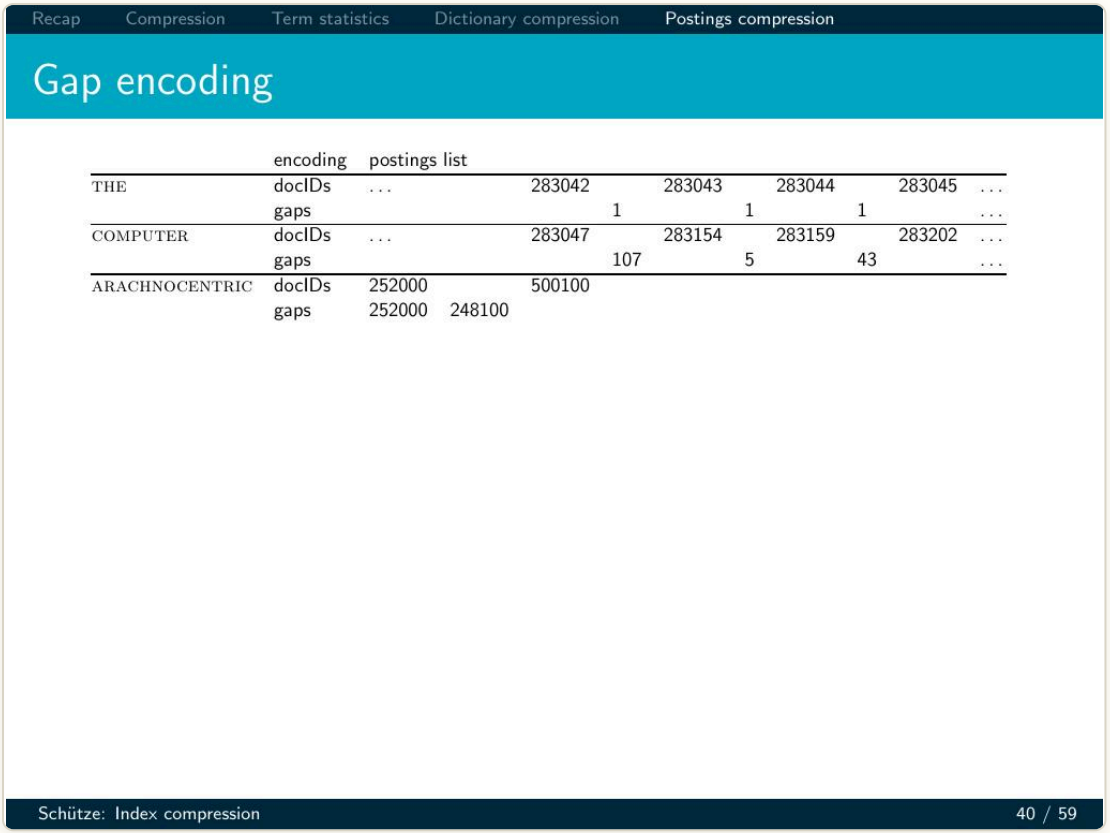


핵심 아이디어: docID 대신 간격(gap)을 저장

- 포스팅 리스트는 docID **오름차순**
- 예: computer: 283154, 283159, 283202, ...
- **간격**만 저장하면 충분: 283159 − 283154 = 5, 283202 − 283159 = 43
- 간격으로 쓴 포스팅: computer: 283154, 5, 43, ...
- **흔한 단어의 간격은 작다**
- 그래서 작은 간격은 20비트보다 **적은 비트**로 인코딩할 수 있다

✍ 메모

★ 시험 첫 docID는 그대로, 이후는 차이(gap)

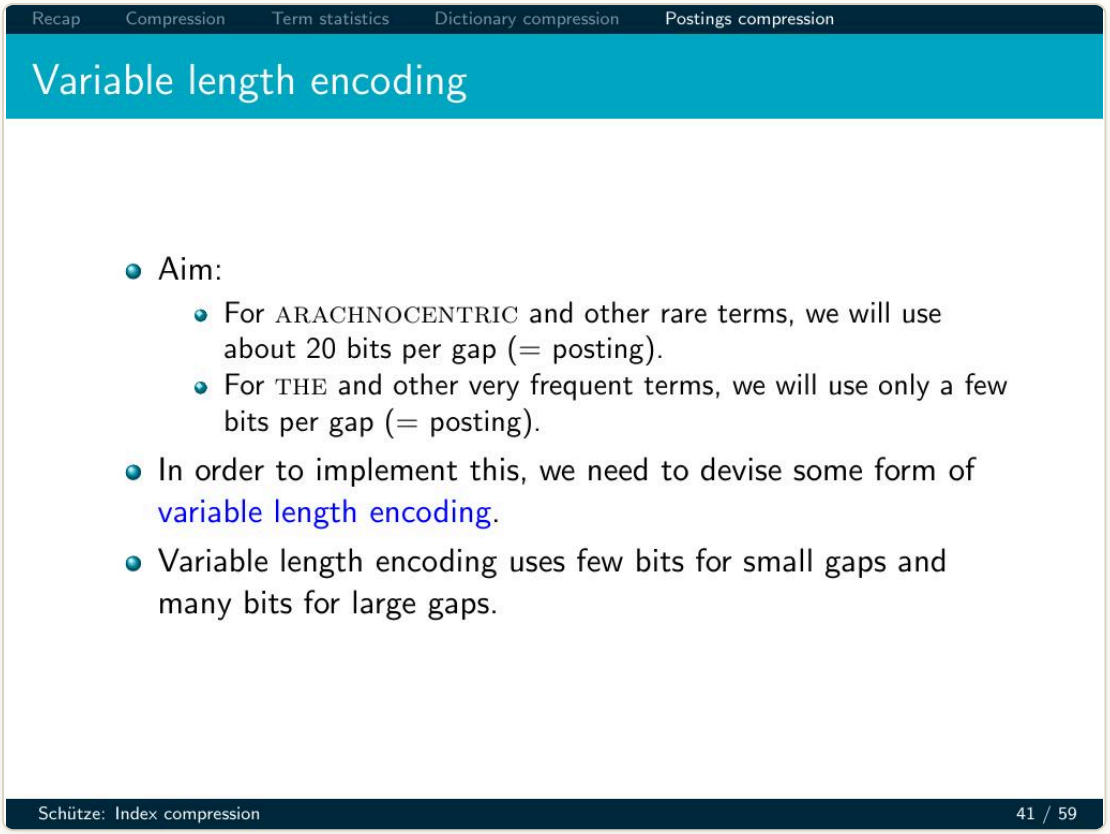


간격 인코딩 (표)

- THE: docID 283042, 283043, 283044, 283045 → 간격 1, 1, 1
- COMPUTER: docID 283047, 283154, 283159, 283202 → 간격 107, 5, 43
- ARACHNOCENTRIC: docID 252000, 500100 → 간격 252000, 248100

✍️ 메모

💡 이해 흔한 단어(the) = 간격 1 / 드문 단어 = 간격이 커서 압축 효과 적음

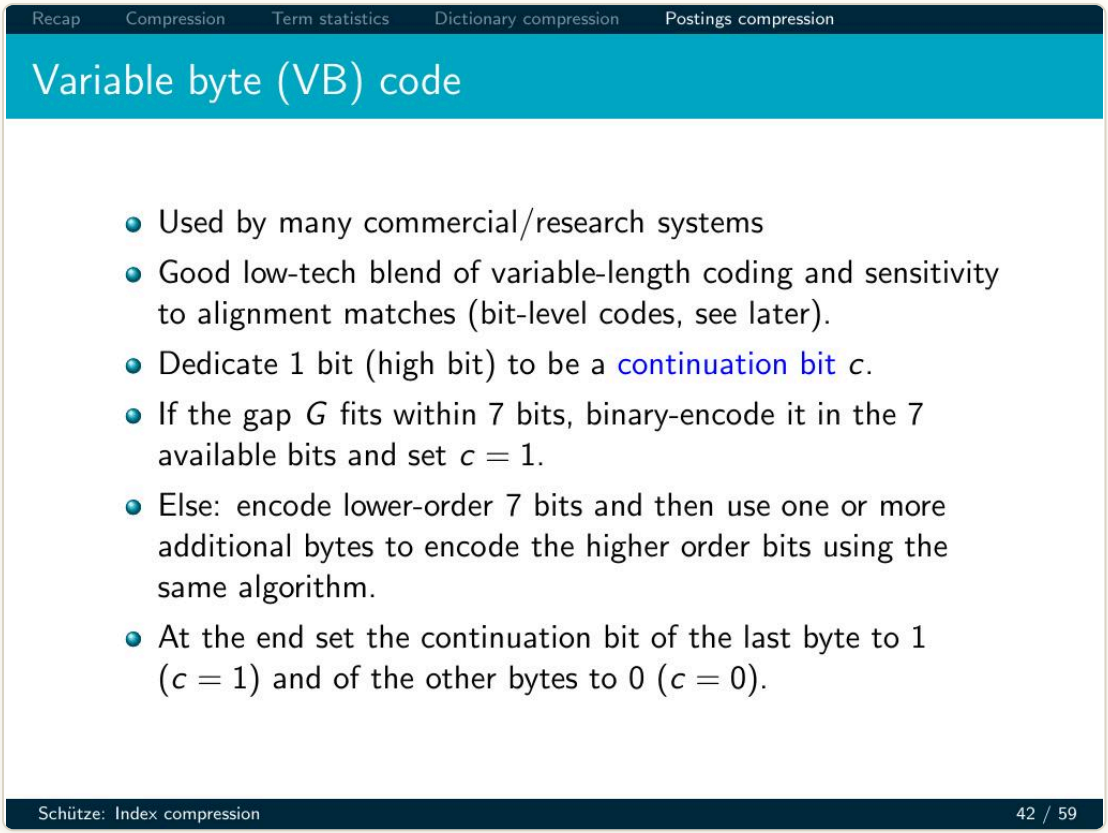


가변 길이 인코딩

- 목표
  - arachnocentric 같은 드문 단어엔 간격(포스팅)당 약 **20비트**
  - the 같은 아주 흔한 단어엔 간격당 **몇 비트만**
- 이를 위해 **가변 길이 인코딩**이 필요
- 작은 간격엔 적은 비트, 큰 간격엔 많은 비트

✍ 메모

— 넘어가도 되는 쪽

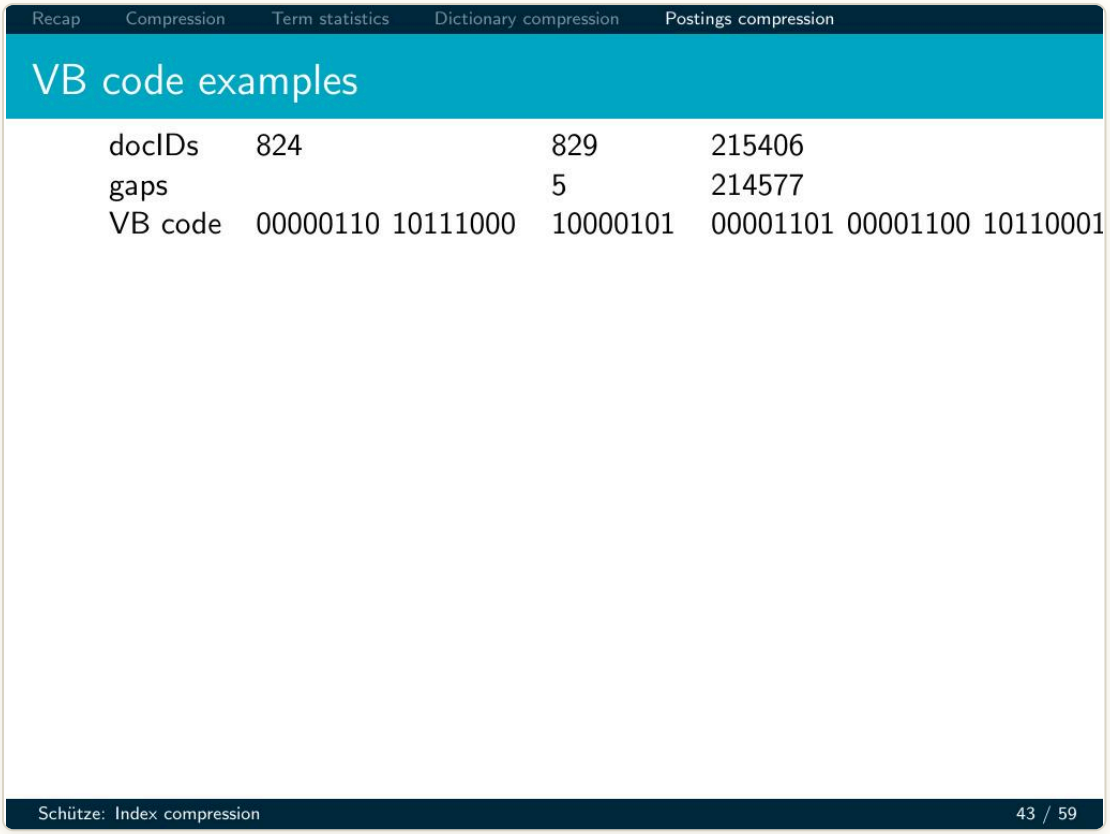


가변 바이트 (VB) 코드

- 많은 상용·연구 시스템이 사용
- 가변 길이 코딩과 **바이트 정렬**을 잘 섞은 단순한 방법 (비트 단위 코드는 뒤에서)
- 각 바이트의 **최상위 1비트를 연속 비트 c**로 쓴다
- 간격 G가 **7비트에 들어가면** 7비트에 이진 인코딩하고 **c = 1**
- 아니면 하위 7비트를 인코딩하고, 상위 비트는 같은 방식으로 추가 바이트에
- **마지막 바이트**의 연속 비트는 **1**, 나머지 바이트는 **0**

✍ 메모

★ 시험 VB: 7비트씩 끊고, 마지막 바이트만 맨 앞 비트 1 (끝 표시)



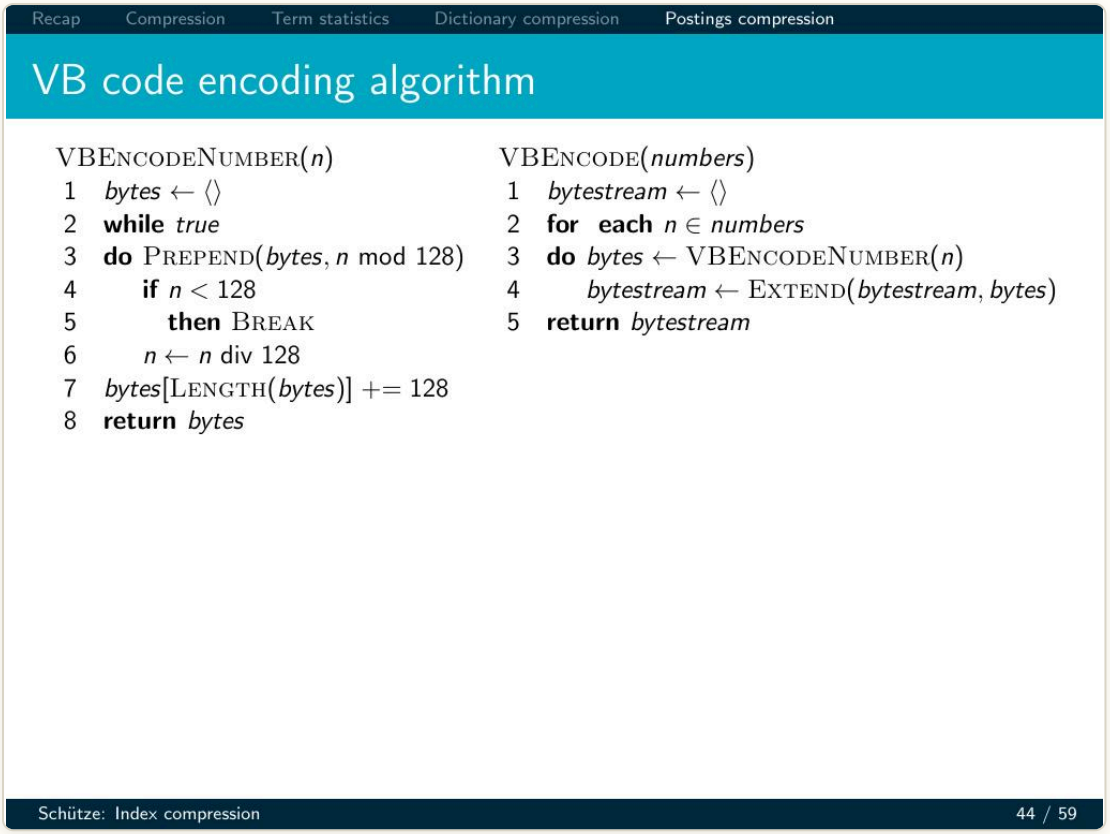
VB 코드 예

- docID: 824, 829, 215406
- 간격: 824, 5, 214577
- VB 코드:

```
824    → 00000110 10111000
5      → 10000101
214577 → 00001101 00001100 10110001
```

✍ 메모

✏ 예·풀이 824 = 6 × 128 + 56 → 0|0000110, 1|0111000

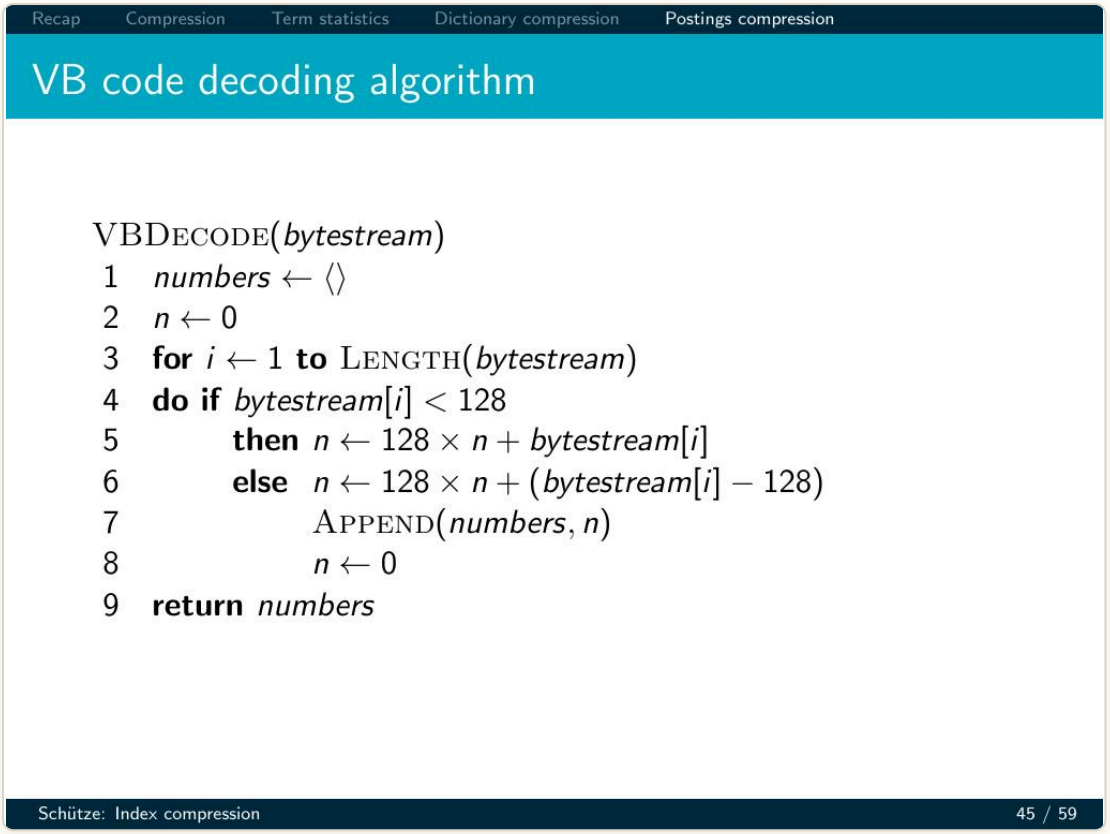


VB 인코딩 알고리즘

```
VBENCODENUMBER(n):
  bytes ← ⟨⟩
  while true: PREPEND(bytes, n mod 128); if n < 128: break; n ← n div 128
  bytes[LENGTH(bytes)] += 128   (마지막 바이트에 연속 비트)
  return bytes
VBENCODE(numbers): 각 n을 VBENCODENUMBER해서 이어 붙임
```

✍ 메모

— 넘어가도 되는 쪽

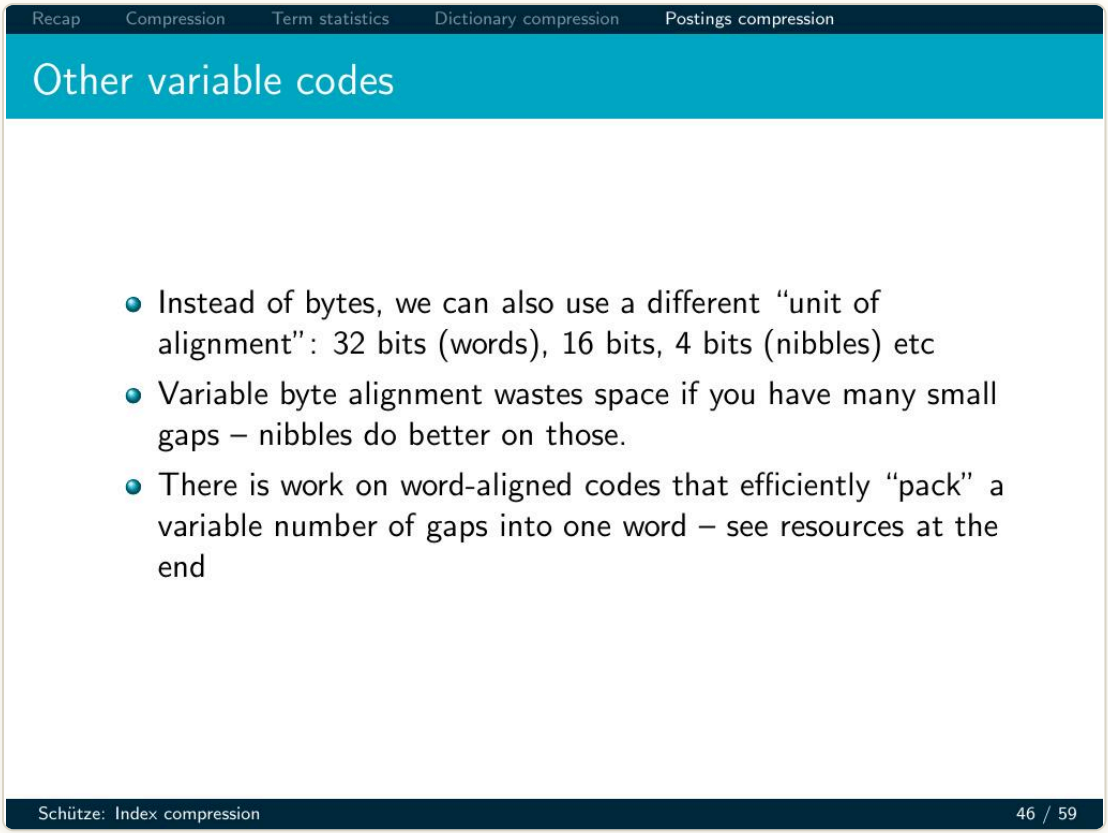


VB 디코딩 알고리즘

```
VBDECODE(bytestream): numbers ← ⟨⟩; n ← 0
for i ← 1 to LENGTH(bytestream):
  bytestream[i] < 128이면: n ← 128 × n + bytestream[i]
  아니면: n ← 128 × n + (bytestream[i] − 128); APPEND(numbers, n); n ← 0
return numbers
```

✍ 메모

— 넘어가도 되는 쪽

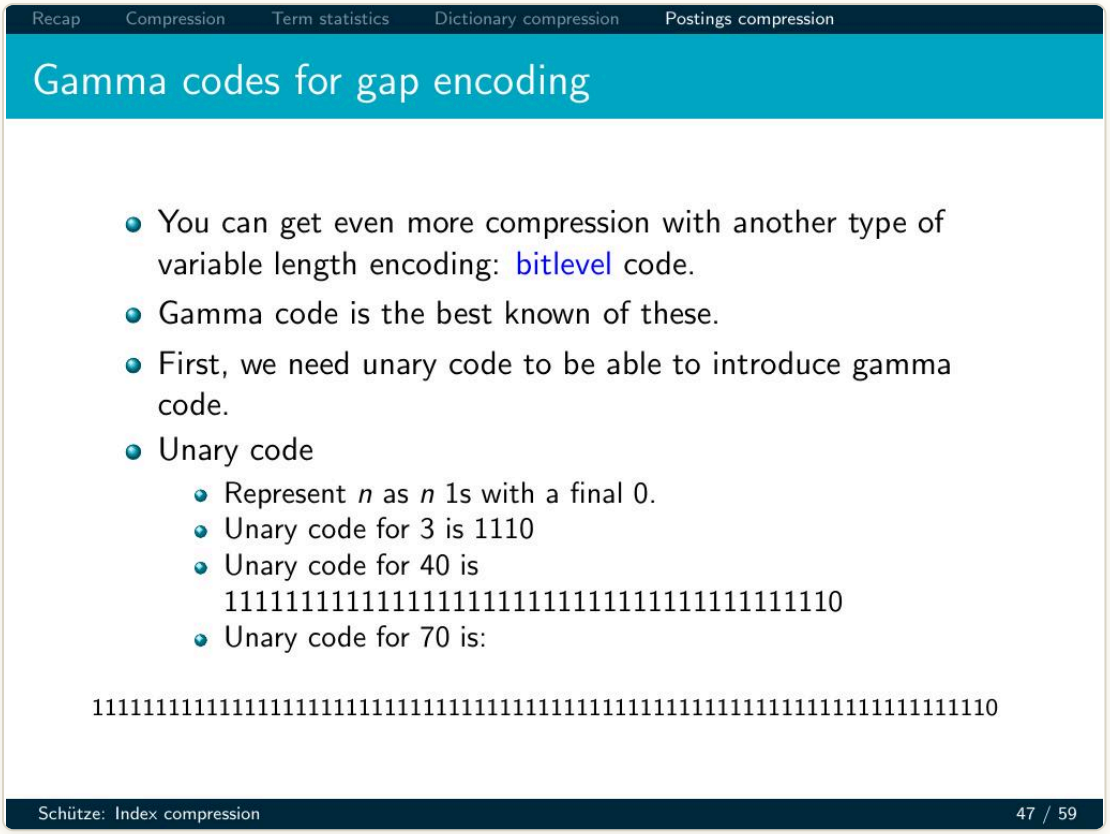


다른 가변 코드

- 바이트 대신 다른 **정렬 단위**도 가능: 32비트(워드), 16비트, 4비트(니블) 등
- 작은 간격이 많으면 가변 바이트는 공간을 **낭비** — 니블이 더 낫다
- 가변 개수의 간격을 워드 하나에 효율적으로 '채워 넣는' 워드 정렬 코드 연구도 있다 (끝의 참고 자료)

✍ 메모

— 넘어가도 되는 쪽

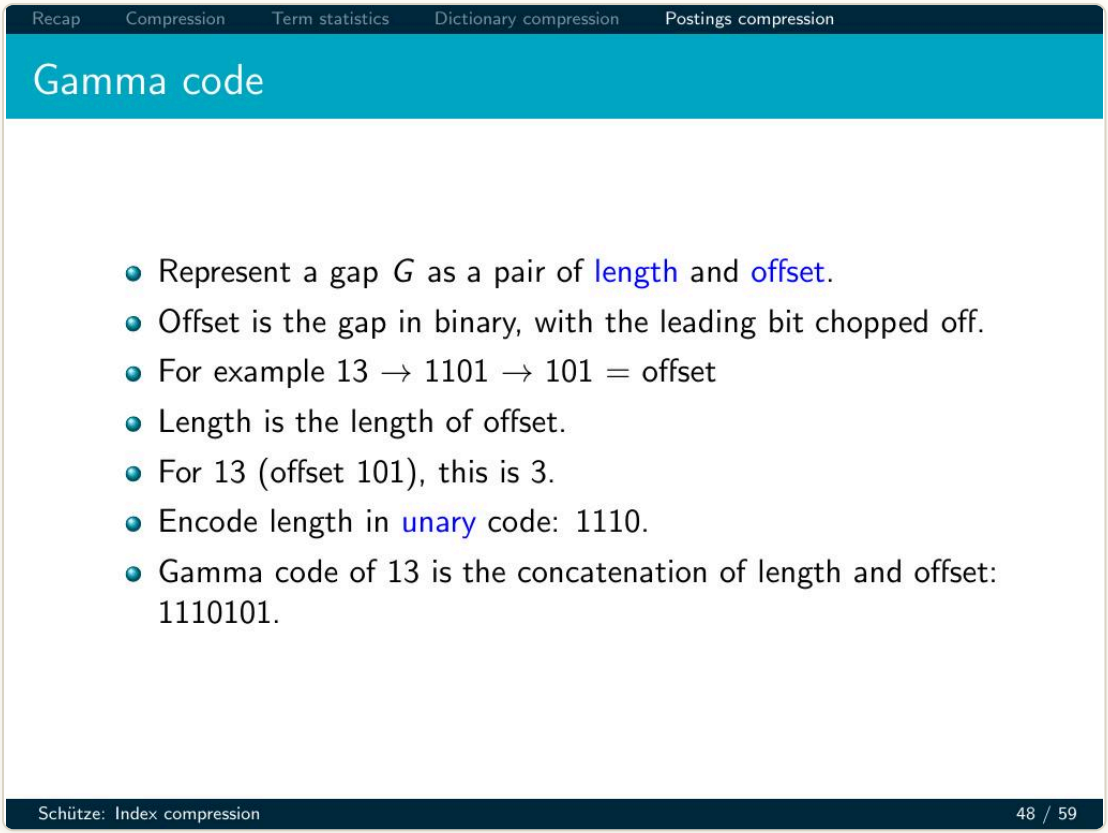


간격 인코딩용 감마 코드

- 다른 종류의 가변 길이 인코딩인 **비트 단위 코드**로 더 압축할 수 있다
- **감마 코드**가 가장 잘 알려진 것
- 감마 코드를 소개하려면 먼저 **단항 코드(unary code)**가 필요
- **단항 코드**: n을 **1을 n개 + 끝에 0**으로
  - 3의 단항 코드: **1110**
  - 40의 단항 코드: 1 ×40 + 0
  - 70의 단항 코드: 1 ×70 + 0

✍ 메모

— 넘어가도 되는 쪽

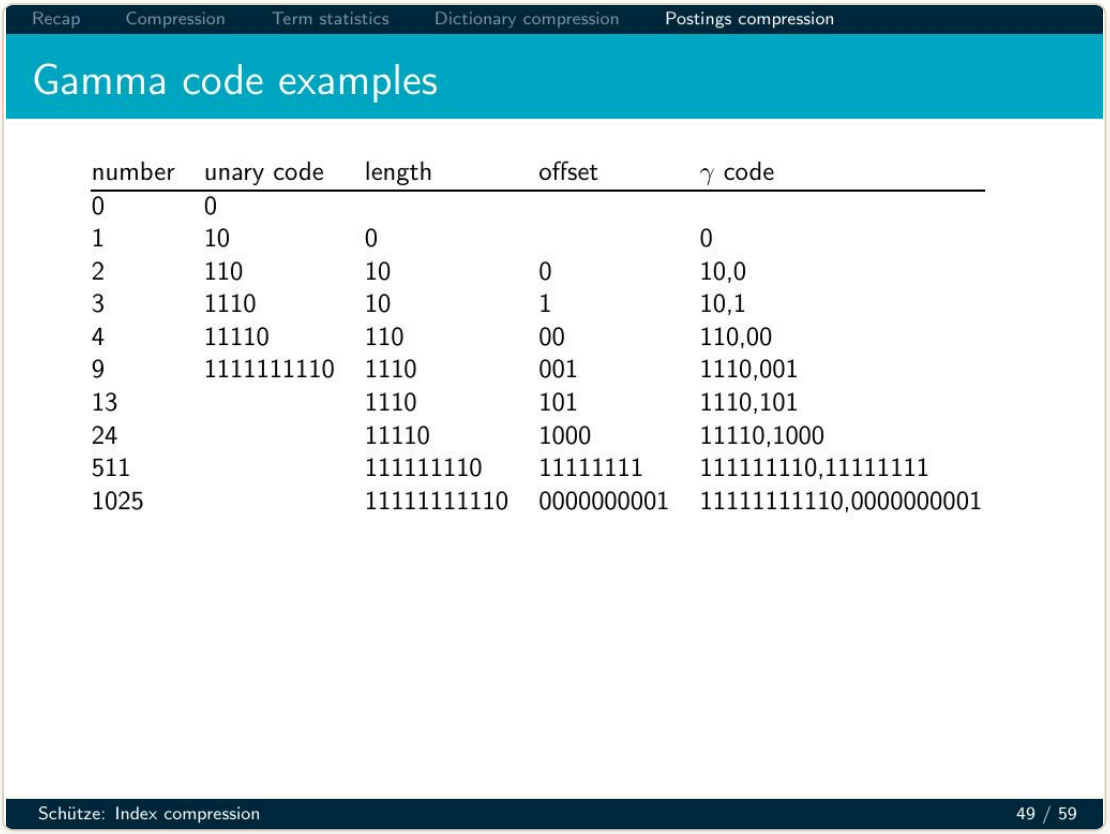


감마 코드 (Gamma code)

- 간격 G를 **(길이, 오프셋)** 쌍으로 표현
- **오프셋** = 간격의 이진수에서 **맨 앞 1비트를 뗀 것**
  - 예: 13 → 1101 → **101** = 오프셋
- **길이** = 오프셋의 길이
  - 13(오프셋 101)이면 **3**
- 길이를 **단항 코드**로: **1110**
- 13의 감마 코드 = 길이와 오프셋을 이어 붙인 것: **1110101**

✍ 메모

★ 시험 감마 = 단항(오프셋 길이) + 오프셋(이진수에서 첫 1 제거)



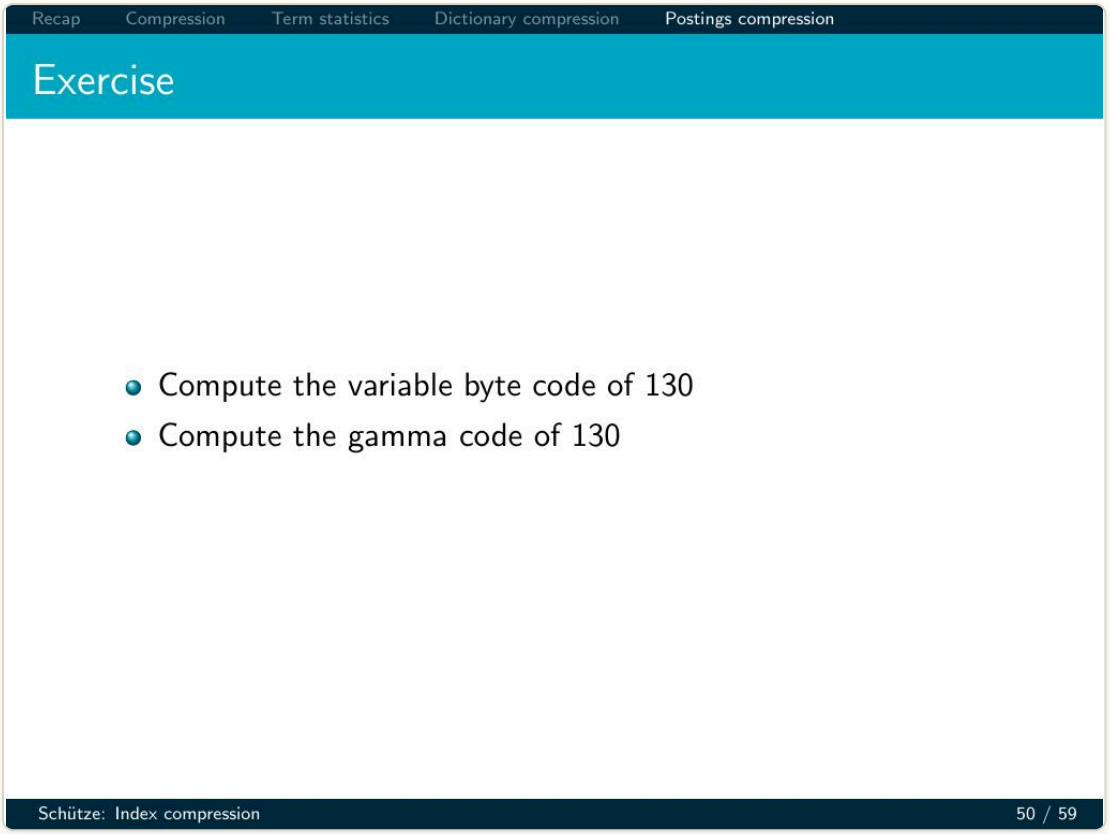
감마 코드 예 (표)

- 수 / 단항 / 길이 / 오프셋 / γ 코드

0	0			
1	10	0	-	0
2	110	10	0	10,0
3	1110	10	1	10,1
4	11110	110	00	110,00
9	1111111110	1110	001	1110,001
13		1110	101	1110,101
24		11110	1000	11110,1000
511		111111110	11111111	111111110,11111111
1025		11111111110	0000000001	11111111110,0000000001

✍ 메모

⚠ 주의 0은 감마 코드로 표현 불가 (간격은 항상 1 이상이라 괜찮다)



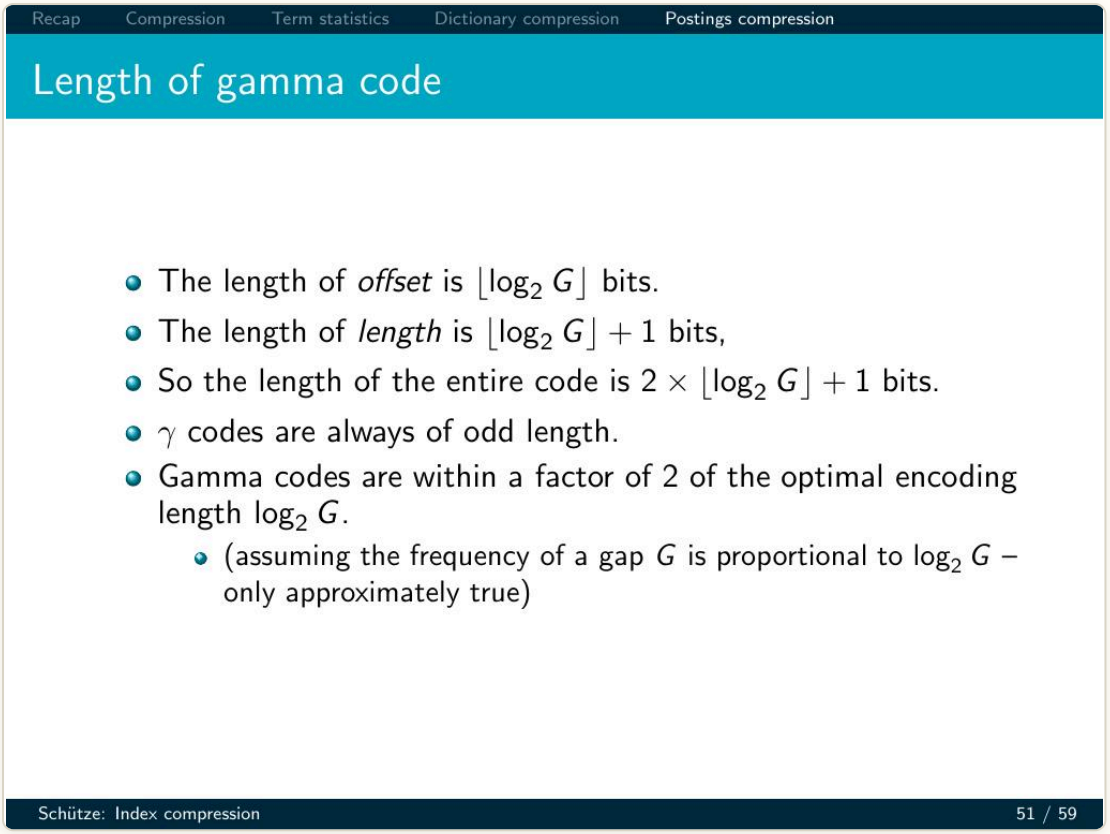
연습

- 130의 **가변 바이트 코드**를 구하라
- 130의 **감마 코드**를 구하라

✍️ 메모

✏ 예·풀이 VB: 130 = 1 × 128 + 2 → **00000001 10000010**

✏ 예·풀이 γ: 130 = 10000010_2 → 오프셋 0000010 (7비트) → 길이 7 = 11111110 → **11111110 0000010**

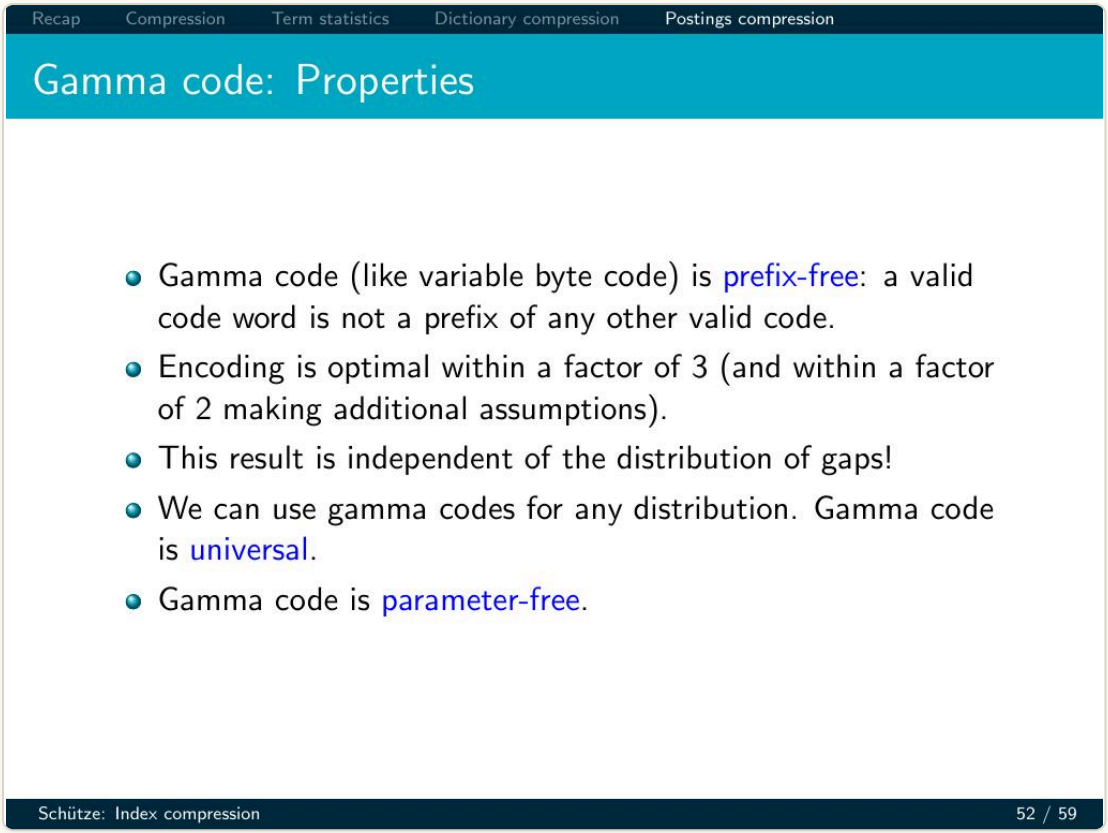


감마 코드의 길이

- 오프셋 길이: **⌊log₂G⌋** 비트
- 길이 부분: **⌊log₂G⌋ + 1** 비트
- 그래서 전체 길이: **2 × ⌊log₂G⌋ + 1** 비트
- γ 코드는 항상 **홀수 길이**
- 감마 코드는 최적 인코딩 길이 $\log_2 G$의 **2배 이내**
  - (간격 G의 빈도가 $\log_2 G$에 비례한다고 가정 — 대략만 맞음)

✍ 메모

★ 시험 길이 = 2⌊log₂G⌋ + 1 (13 → 2×3+1 = 7비트 ✓)

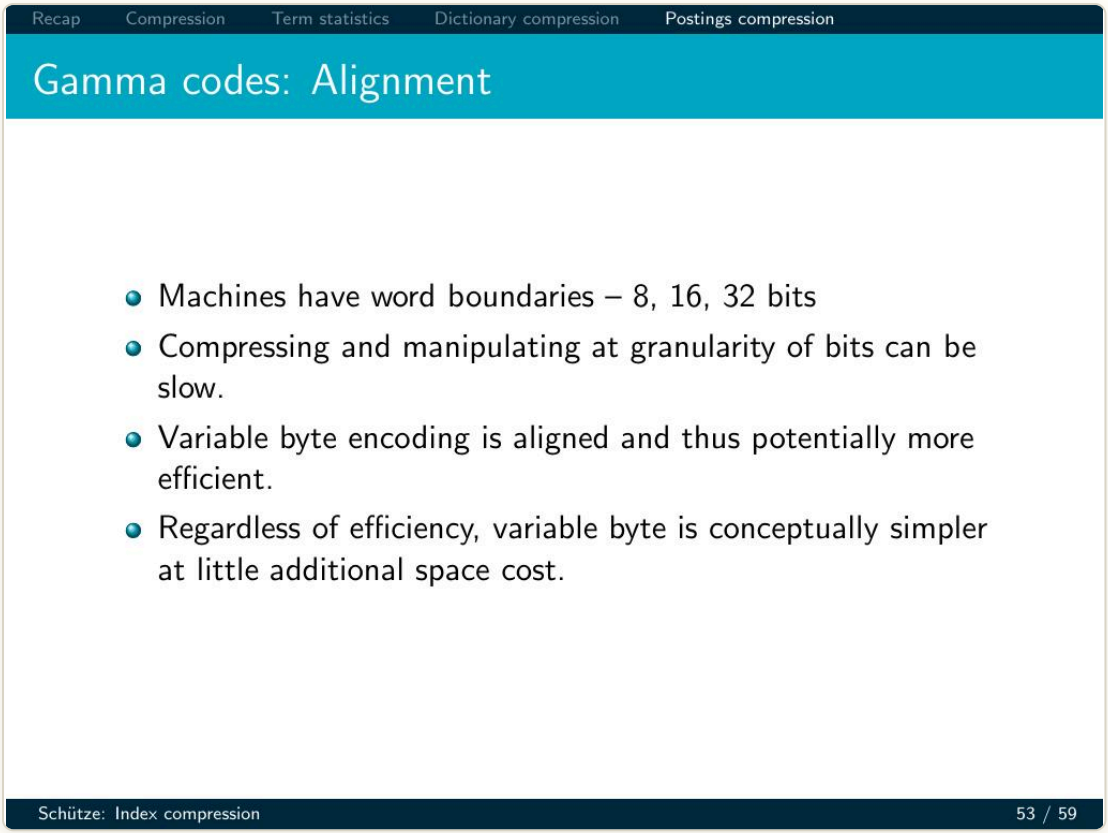


감마 코드: 성질

- 감마 코드는 (VB처럼) **접두어 없음(prefix-free)**: 유효한 코드어가 다른 코드어의 접두어가 아니다
- 인코딩이 최적의 **3배 이내** (추가 가정을 하면 2배 이내)
- 이 결과는 **간격 분포와 무관**하다!
- 어떤 분포에도 감마 코드를 쓸 수 있다 — 감마 코드는 **보편적(universal)**
- 감마 코드는 **매개변수가 없다(parameter-free)**

✍ 메모

★ 시험 성질 4개: prefix-free · 최적의 상수배 · universal · parameter-free

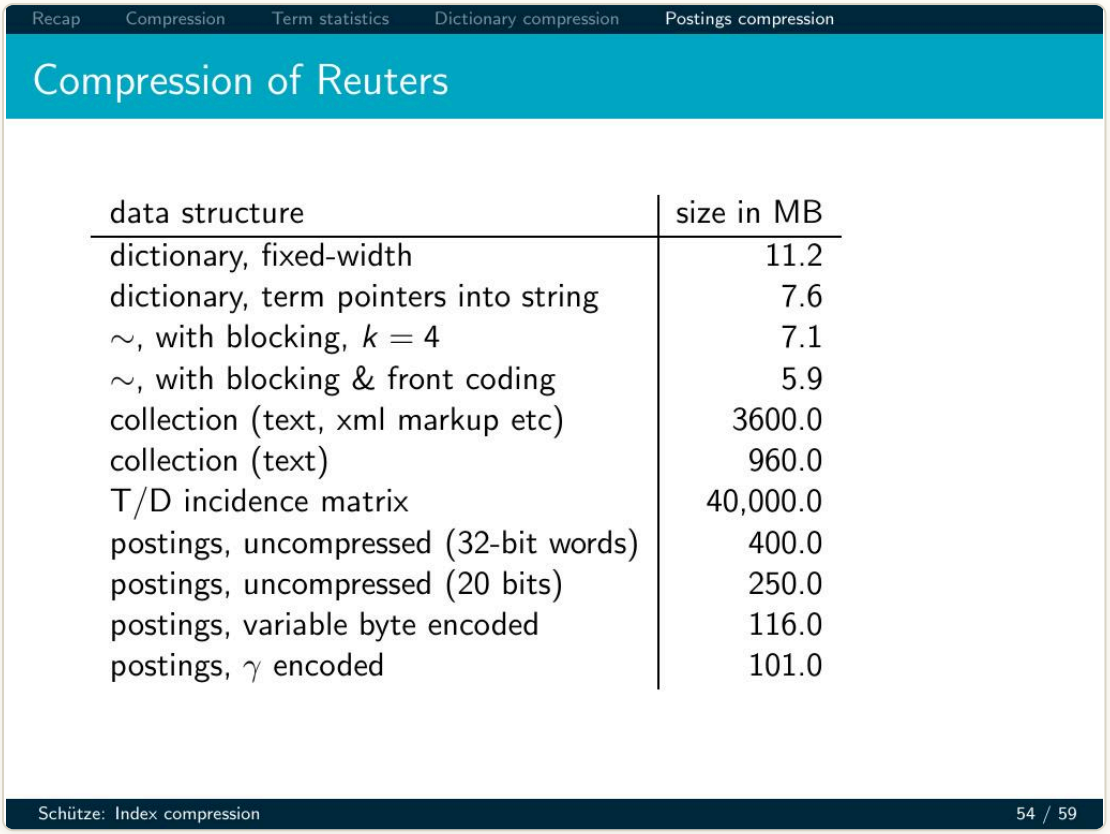


감마 코드: 정렬

- 기계는 워드 경계가 있다 — 8, 16, 32비트
- **비트 단위**로 압축·처리하면 **느릴 수 있다**
- 가변 바이트 인코딩은 **정렬돼 있어** 잠재적으로 더 효율적
- 효율과 별개로 가변 바이트가 **개념적으로 더 단순**하고, 공간 손해도 적다

✍ 메모

★ 시험 감마 = 공간↓ 느림 / VB = 조금 더 크지만 빠르고 단순



로이터 압축 (표)

- 사전 고정 폭 **11.2** / 문자열 포인터 **7.6** / + 블로킹 k=4 **7.1** / + 블로킹·앞부분 코딩 **5.9**
- 컬렉션(텍스트+XML) 3600.0 / 컬렉션(텍스트) 960.0 / 단어-문서 행렬 40,000.0
- 포스팅 무압축(32비트) 400.0 / 무압축(20비트) 250.0 / **가변 바이트 116.0** / **감마 101.0** (단위 MB)

✍ 메모

— 넘어가도 되는 쪽

Term-document incidence matrix

	Anthony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	...
ANTHONY	1	1	0	0	0	1	
BRUTUS	1	1	0	1	0	0	
CAESAR	1	1	0	1	1	1	
CALPURNIA	0	1	0	0	0	0	
CLEOPATRA	1	0	0	0	0	0	
MERCY	1	0	1	1	1	1	
WORSER	1	0	1	1	1	0	
...							

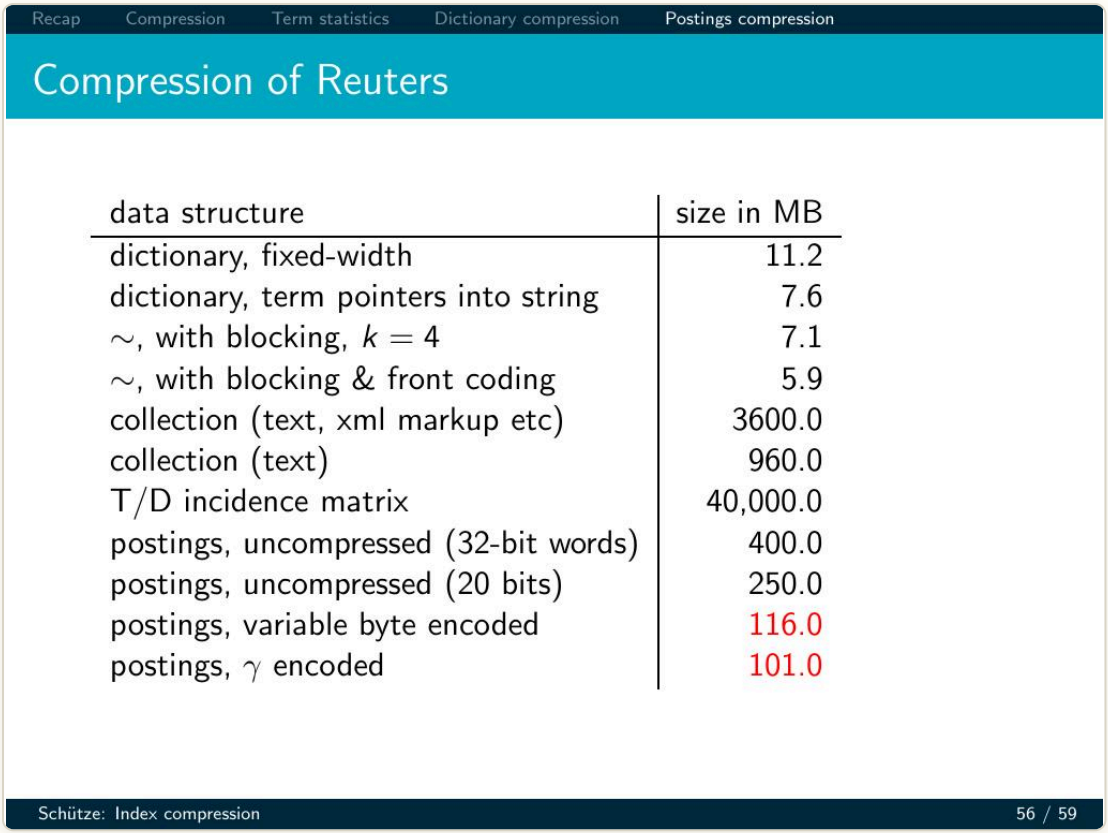
Entry is 1 if term occurs. Example: CALPURNIA occurs in *Julius Caesar*.
Entry is 0 if term doesn't occur. Example: CALPURNIA doesn't occur in *The tempest*.

단어-문서 발생 행렬 (복습)

- 1강의 0/1 행렬 — 로이터라면 40,000MB가 필요 (400,000 × 800,000 비트)

✍ 메모

✏ 예·풀이 $4\times10^5 \times 8\times10^5 = 3.2\times10^{11}$ 비트 = 4×10^{10} 바이트 = 40,000MB

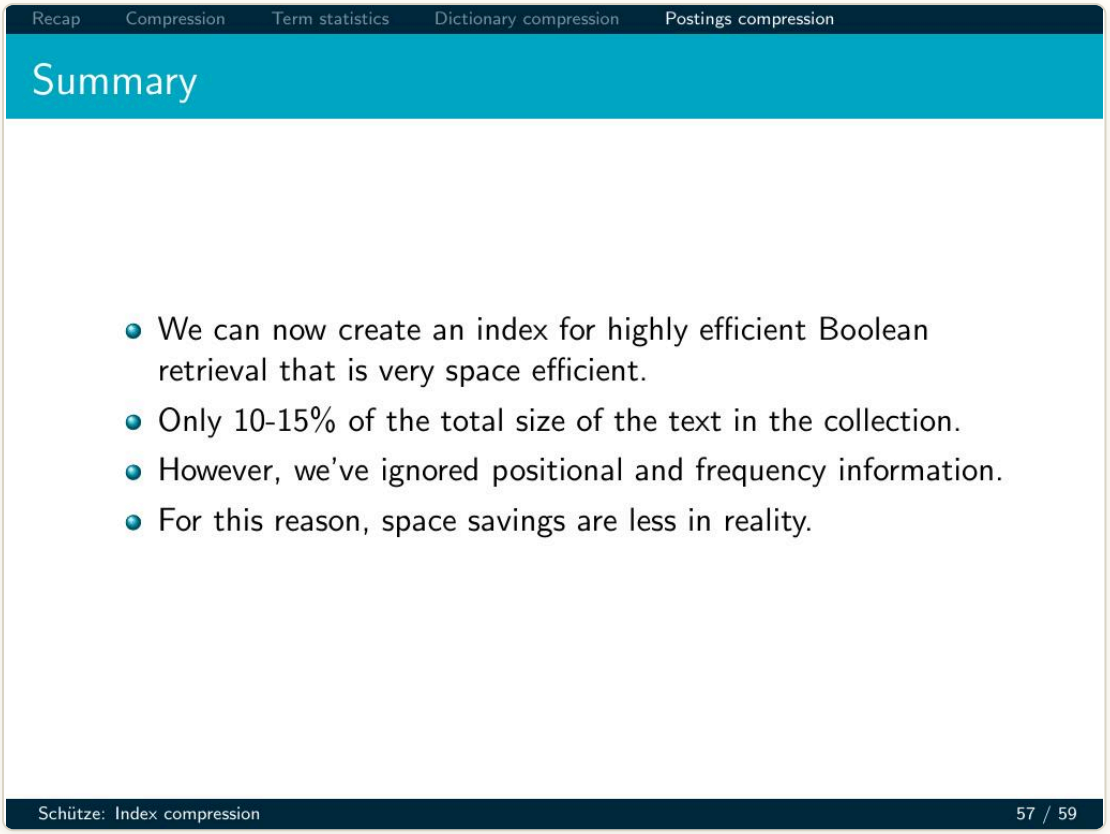


로이터 압축 (표, 강조)

- 사전 고정 폭 **11.2** / 문자열 포인터 **7.6** / + 블로킹 k=4 **7.1** / + 블로킹·앞부분 코딩 **5.9**
- 컬렉션(텍스트+XML) 3600.0 / 컬렉션(텍스트) 960.0 / 단어-문서 행렬 40,000.0
- 포스팅 무압축(32비트) 400.0 / 무압축(20비트) 250.0 / **가변 바이트 116.0** / **감마 101.0** (단위 MB)

✍ 메모

★ 시험 포스팅: 400 → 250 → VB 116 → γ 101 MB

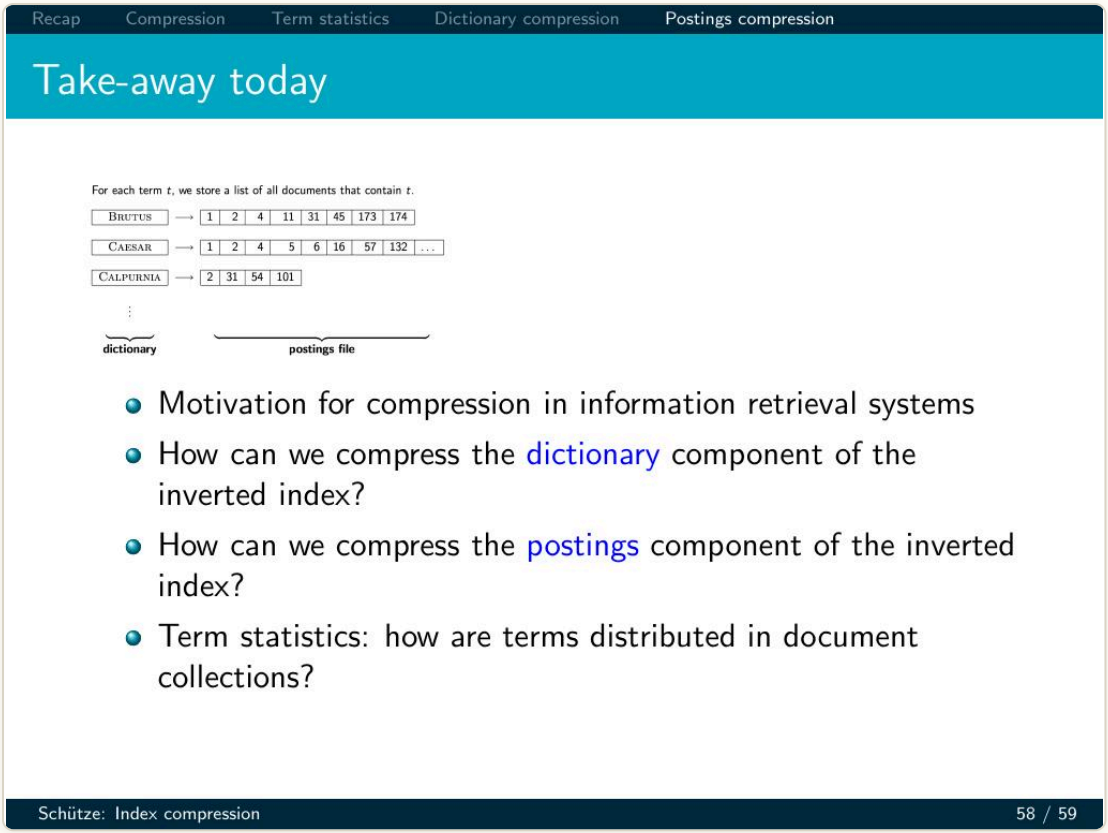


요약

- 이제 매우 효율적인 불리언 검색용 색인을 **아주 작은 공간**으로 만들 수 있다
- 컬렉션 텍스트 전체 크기의 **10~15%**뿐
- 하지만 **위치·빈도 정보는 무시**했다
- 그래서 실제 절약은 이보다 적다

✍ 메모

— 넘어가도 되는 쪽

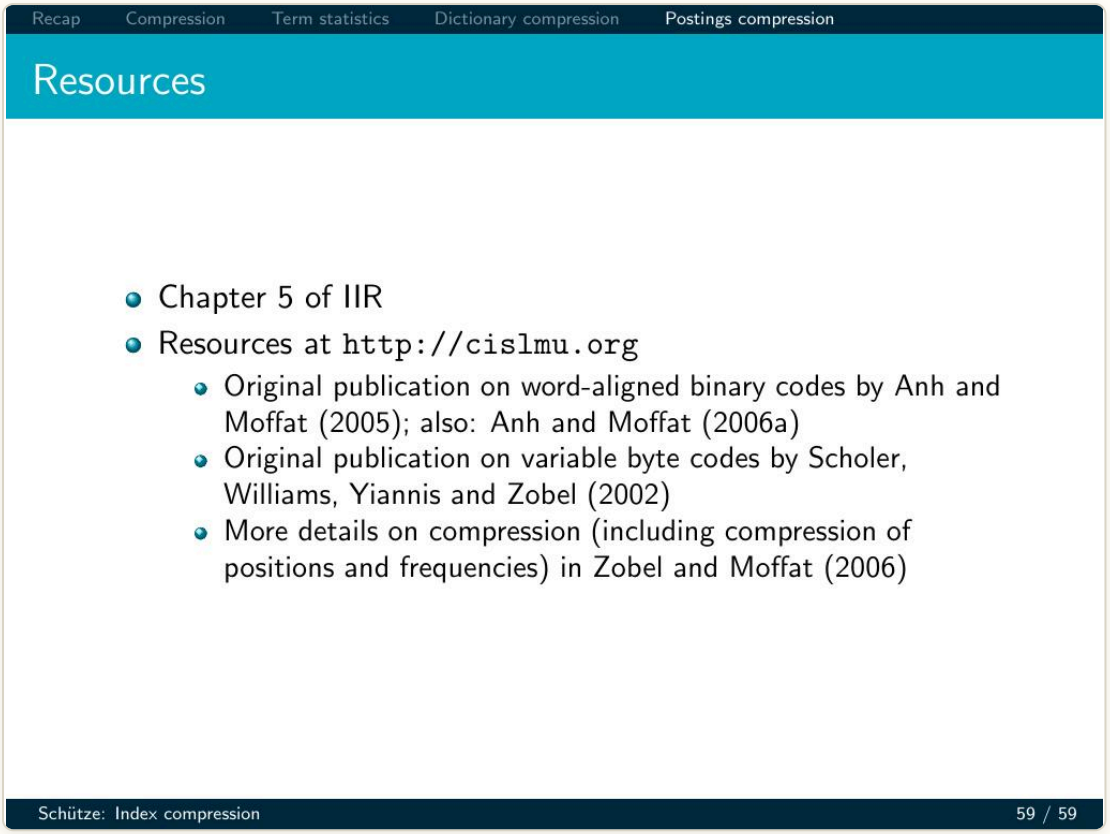


오늘 가져갈 것

- IR에서 압축의 동기
- 사전 압축 (문자열, 블로킹, 앞부분 코딩)
- 포스팅 압축 (간격, VB, 감마)
- 단어 통계 (힙스, 지프)

✍ 메모

— 넘어가도 되는 쪽



참고 자료

- IIR 5장, http://cislmu.org
- 워드 정렬 이진 코드: Anh & Moffat (2005, 2006a)
- 가변 바이트 코드: Scholer, Williams, Yiannis & Zobel (2002)
- 압축 상세(위치·빈도 압축 포함): Zobel & Moffat (2006)

✍ 메모

— 넘어가도 되는 쪽